ADITYA

COLLEGE OF ENGINEERING & TECHNOLOGY

An AUTONOMOUS Institution

Approved by AICTE, Permanently Affiliated to JNTUK,
Accredited by NBA & NAAC with A+ Grade
Recognized by UGC under Section 2(f) and 12(B) of UGC Act, 1956
Aditya Nagar, ADB Road, Surampalem, Kakinada District - 533437, A.P.

Department of

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Name : Roll No. : | | | | | | | | | | |

Certified that this is the bonafide record of practical work done by

Mr. /Ms. .....

a student of .....with PIN No. .....

in the ..... Laboratory during the year .....

No. of Practicals Conducted : | | No. of Practicals Attended : | |

Signature - Faculty Incharge **Signature - Head of the Department**

Submitted for the Practical examination held on .....

EXAMINER - 1 **EXAMINER - 2**

ADITYA COLLEGE OF ENGINEERING AND TECHNOLOGY

INSTITUTE VISION AND MISSION

VISION:

To induce higher planes of learning by imparting technical education with

- ✓ International standards
- ✓ Applied research
- ✓ Creative Ability
- ✓ Value based instruction and to emerge as a premiere institute

MISSION:

Achieving academic excellence by providing globally acceptable technical education by forecasting technology through

- ✓ Innovative Research And development
- ✓ Industry Institute Interaction
- ✓ Empowered Manpower

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT VISION AND MISSION

VISION:

To become a center for excellence in Computer Science and Engineering education and innovation.

MISSION:

- Provide state of art infrastructure
- Adapt skill-based learner centric teaching methodology
- Organize socio cultural events for better society
- Undertake collaborative works with academia and industry
- Encourage students and staff self-motivated, problem-solving individuals using Artificial Intelligence
- Encourage entrepreneurship in young minds.

Compute Networks Lab

List of Experiments:

1. Study of Network devices in detail and connect the computers in Local Area Network.
2. Write a Program to implement the datalink layer farming methods such as
   i) Character stuffing ii) bit stuffing.
3. Write a Program to implement datalink layer farming method checksum.
4. Write a program for Hamming Code generation for error detection and correction.
5. WriteaProgramtoimplementonadatasetofcharactersthethree CRCpolynomials –CRC12,CRC 16 and CRC CCIP.
6. Write a Program to implement Sliding window protocol for Go-back-N.
7. Write a Program to implement Sliding window protocol for Selective repeat.
8. Write a Program to implement Stop and Wait Protocol.
9. Write a program for congestion control using leaky bucket algorithm
10. Write a Program to implement Dijkstra's algorithm to compute the Shortest path through a graph.
11. Write a Program to implement Distance vector routing algorithm by obtaining routing table at each node (Take an example subnet graph with weights indicating delay between nodes).
12. Write a Program to implement Broadcast tree by taking subnet of hosts.
13. Wireshark
    i. Packet Capture Using Wireshark
    ii. Starting Wireshark
    iii. Viewing Captured Traffic
    iv. Analysis and Statistics & Filters.
14. How to run Nmapscan
15. Operating System Detection using Nmap
16. Do the following using NS2 Simulator
    i. NS2 Simulator-Introduction
    ii. Simulate to Find the Number of Packets Dropped
    iii. Simulate to Find the Number of Packets Dropped by TCP/UDP
    iv. Simulate to Find the Number of Packets Dropped due to Congestion
    v. Simulate to Compare DataRate &Throughput.

Pointer

S No	Date	Name of the Experiment	Page No	Remark

Pointer

S No	Date	Name of the Experiment	Page No	Remark

Computer Networks Lab

EXPERIMENT - 1

1. StudyofNetworkdevicesindetailandconnectthecomputersinLocalAreaNetwork.

Aim: Study of following Network Devices in Detail

- Repeater
- Hub
- Switch
- Bridge
- Router
- Gate Way

Apparatus (Software): No software or hardware needed.

Procedure: Following should be done to understand this practical.

1. **Repeater:**Functioning at Physical Layer.A **repeater** is an electronic device that receives a signal and retransmits it at a higher level and/or higher power, or onto the other side of an obstruction, so that the signal can cover longer distances. Repeater have two ports ,so cannot be use to connect for more than two devices

2. **Hub:** An **Ethernet hub**, **active hub**, **network hub**, **repeater hub**, **hub** or **concentrator** is a device for connecting multiple twisted pair or fiber optic Ethernet devices together and making them act as a single network segment. Hubs work at the physical layer (layer 1) of the OSI model. The device is a form of multiport repeater. Repeater hubs also participate in collision detection, forwarding a jam signal to all ports if it detects a collision.

3. **Switch:**A **network switch** or **switching hub** is a computer networking device that connects network segments.The term commonly refers to a network bridge that processes and routes data at the data link layer (layer 2) of the OSI model. Switches that additionally process data at the network layer (layer 3 and above) are often referred to as Layer 3 switches or multilayer switches.

4. **Bridge:** A **network bridge** connects multiple network segments at the data link layer (Layer 2) of the OSI model. In Ethernet networks, the term *bridge* formally means a device that behaves according to the IEEE 802.1D standard. A bridge and switch are very much alike; a switch being a bridge with numerous ports. *Switch* or *Layer 2 switch* is often used interchangeably with *bridge*.Bridges can analyze incoming data packets to determine if the bridge is able to send the given packet to another segment of the network.

5. **Router:** A **router** is an electronic device that interconnects two or more computer networks, and selectively interchanges packets of data between them. Each data packet contains address information that a router can use to determine if the source and destination are on the same network, or if the data packet must be transferred from one network to another. Where multiple routers are used in a large collection of interconnected networks, the routers exchange information about target system addresses, so that each router can build up a table showing the preferred paths between any two systems on the interconnected networks.

6. **Gate Way:** In a communications network, a network node equipped for interfacing with another network that uses different protocols.

- A gateway may contain devices such as protocol translators, impedance matching devices, rate converters, fault isolators, or signal translators as necessary to provide system interoperability. It also requires the establishment of mutually acceptable administrative procedures between both networks.
- A protocol translation/mapping gateway interconnects networks with different network protocol technologies by performing the required protocol conversions.

EXPERIMENT - 2

2. Write a Program to implement the datalink layer farming methods such as
i) Character stuffing ii) bit stuffing.

i)Character stuffing

```
#include<stdio.h>
#include<string.h>
main()
{
 char a[30],fs[50]="",t[3],sd , ed ,
 x[3],s[3],d[3],y[3]; int i, j, p = 0, q = 0;
 printf("Enter characters to be
 stuffed:"); scanf("%s", a);
 printf("\n Enter a character that represents starting
 delimiter:"); scanf(" %c", & sd);
 printf("\n Enter a character that represent sending
 delimiter:"); scanf(" %c", & ed);
 x[0]= s[0]= s[1]=sd;
 x[1]=s[2]='\0';
 y[0]=d[0] =d[1]=ed;
 d[2]= y[1]='\0';
 strcat(fs,x);
 for (i = 0; a[i] != '\0'; i++){
  t[0] = a[i];
  t[1]= '\0';
  if (t[0] == sd)
  strcat(fs, s);
  else if(t[0]==ed)
   strcat(fs, d);
  else
  strcat
   (fs,t)
   ;
 }
 strcat(fs, y);
 printf("\n After
 stuffing:%s",fs); getch();
}
```

OUTPUT:

Enter character to be stuffed : ADITYA
Enter character that represents starting delimiter : D
Enter character that represents ending delimiter : S
After stuffing : DADITYAS

ii)bit stuffing

```
#include<stdio.h>
#include<string.h>

int main() {
    int a[20], b[30], i, j, k, count, n;
    printf("Enter frame size in bits:");
    scanf("%d", & n);
    printf("Enter the frame in the form of 0 and 1:");
    for (i = 0; i < n; i++)
    scanf("%d", & a[i]);
    i =0;
    count=1;
    j =0;
    while (i<n){
     if (a[i] == 1 && count ==5){ b[j] = a[i];
       for(k=i+1;a[k]==1&&k<n&&count<5;k++){
        j++;
         b[j]=a[k]; count++;
         if (count == 5){
         j++;
           b[j]= 0;
         }
         i=k;
       }
      }else{
        b[j]=a[i];
      } i++; j++;
    }
    printf("AfterBitStuffing:");
    printf("0 1 1 1 1 1 1 0 ");
    for (i = 0; i < j; i++)
    printf("%d ", b[i]);
    printf("0 1111 110");
    return0;
}
```

OUTPUT:

```
Enter frame size : 8
Enter the frame  : 0
1
1
1
1
1
1
0

After bit stuffing:011111010
```

EXPERIMENT - 3

3. Write a Program to implement datalink layer farming method checksum.

```
#include<stdio.h>
#include<string.h>

int main()
{
   char a[20],b[20];
   charsum[20],compleme
   nt[20]; int i,length;

printf("Enter first binary string\n");
   scanf("%s",&a);
   printf("Enter second binary
   string\n"); scanf("%s",&b);
   if(strlen(a) == strlen(b){
    length = strlen(a);
char carry='0';

for(i=length-1;i>=0;i--)
      {
if(a[i] == '0' && b[i] == '0' && carry == '0')
         {
            sum[i]='0';
            carry='0';
         }
         elseif(a[i]=='0'&&b[i]=='0'&&carry=='1')
         {
            sum[i]='1';
            carry='0';
          }
         elseif(a[i]=='0'&&b[i]=='1'&&carry=='0')
         {
            sum[i]='1';
            carry='0';
          }
         elseif(a[i]=='0'&&b[i]=='1'&&carry=='1')
         {

            sum[i]='0';
            carry='1';
```

```
        }
      elseif(a[i] == '1' && b[i] == '0' && carry == '0')
      {
        sum[i]='1';
        carry='0';
      }
      elseif(a[i]=='1'&&b[i]=='0'&&carry=='1')
      {
        sum[i]='0';
        carry='1';
       }
      elseif(a[i]=='1'&&b[i]=='1'&&carry=='0')
      {
        sum[i]='0';
        carry='1';
       }
      elseif(a[i] == '1' && b[i] == '1' && carry == '1')
      {
        sum[i]='1';
        carry='1';
       }
      else
        break;
    }
printf("\nSum=%c%s",carry,sum);
for(i=0;i<length;i++)
    {
      if(sum[i]=='0')
        complement[i]='1';
      else
        complement[i]='0';
    }
    if(complement
      [length –
      1]==’1’){
    carry='0';
    else
      carry='1';
     printf("\nChecksum=%c%s",carry,complement);
       }
else{
printf("\nWronginputstrings");
```

```
}
}
```

OUTPUT:

```
Enter first binary string
101100
Enter second binary string
110011
Sum=101111
Checksum=0100000
```

EXPERIMENT - 4

4.Write a program for hamming code generation for error detection and correct.

```
#include <math.h>
#include<stdio.h>
int input[32];
int code[32];
int ham_calc(int, int);
void solve(int input[],int);
int ham_calc(int position,int c_l)
{
int count= 0, i, j;
i = position - 1;
while (i < c_l) {
for( j = i; j < i + position; j++) {
if (code[j] == 1)
count++;
}
i=i+2*position;
}
if ( count % 2 == 0)
return 0;
else
return 1;
}
void solve(int input[], int n)
{
int i, p_n=0,c_l,j,k;
i = 0;
while(n>(int)pow(2,i)-(i+1)){
p_n++;
i++;
}
c_l=p_n+n;
j = k = 0;
for( i=0; i<c_l; i++ ){
if(i==((int)pow(2,k)-1)){
code[i] = 0;
k++;
}
else {
code[i] = input[j];
j++;
}
```

```
}
for(i=0; i<p_n; i++) {
int position=(int)pow(2,i);
int value= ham_calc(position,c_l);
code[position-1]=value;
}
printf("\n The generated Code Word is:");
for (i = 0; i < c_l; i++) {
printf("%d",code[i]);
}
}
int main()
{
int N,i;
printf("enter number of bits:");
scanf("%d",&N);
for(i=0; i<N; i++)
{
printf("enter message bit %d:",i);
}
solve(input,N);
return 0;
}
```

OUTPUT:

```
enter number of bits:7
enter message bit 0:1
enter message bit 1:0
enter message bit 2:1
enter message bit 3:1
enter message bit 4:0
enter message bit 5:1
enter message bit 6:1

The generated Code Word is:11100110011.
```

EXPERIMENT - 5

5. Write a program to implement on a data set of characters the three CRC polynomials-CRC 12, CRC 16 and CRC CCIP.

```
#include<stdio.h>
 #include<math.h>
#include<string.h>
 main()
{
int i,j,k,m,n,cl;
char a[10],b[100],c[100];
printf("\n ENTER POLYNANOMIAL:");
scanf("%s",a);
printf("\nENTER THE FRAME:");
scanf("%s",b);
m=strlen(a);
n=strlen(b);
for(i=0;i<m;i++)
{
if(a[i]=='1')
{
m=m-i;
break;
}
}
for(k=0;k<m;k++)
a[k]=a[k+i];
cl=m+n-1;
for(i=0;i<n;i++)
c[i]=b[i];
for(i=n;i<cl;i++)
c[i]='0';
c[i]='\0';
for(i=0;i<n;i++)
 if(c[i]=='1')
{
for(j=i,k=0;k<m;k++,j++)
 if(a[k]==c[j])
 c[j]='0';
 else
 c[j]='1';
}
for(i=0;i<n;i++)
c[i]=b[i];
```

```
 printf("\n THE MESSAGE IS: %s",c);
 return 0;
}
```

OUTPUT:

ENTER POLYNANOMIAL: 10011
ENTER THE FRAME: 1101011111

THE MESSAGE IS: 11010111110010

EXPERIMENT - 6

6. Write a program to implement sliding window protocol for GO-BACK-N.

```
#include<stdio.h>
int main()
{
int windowsize, sent=0, ack, i, c;
 printf("Enter window size: \n");
 scanf("%d",&windowsize);
  while(1)
{
for(i=0; sent <windowsize; i++)
{
printf("Frame %d has been transmitted.\n", sent);
sent++;
if ( sent==windowsize )
break;
}
printf("\n Please enter your choice:\n1-for lost data or ack or delay ack \n2- for exit.\n choice:");
scanf("%d",&c);
if(c== 1)
{
printf("\n enter the frame number:");
scanf("%d",&ack);
for(i=ack;i<windowsize;i++)
{
printf("Frame %d has been retransmitted.\n",i);
 if(i == windowsize)
break;
}
}
else
return 0;
}
return 0;
}
```

OUTPUT:

```
enter window size: 3
Frame 0 has been transmitted.
Frame 1 has been transmitted.
Frame 2 has been transmitted.

 Please enter your choice:
1-for lost data or ack or delay ack
2- for exit
 choice:1

 enter the frame number:2
Frame 2 has been retransmitted.

 Please enter your choice:
1-for lost data or ack or delay ack
2- for exit
  choice:2
```

EXPERIMENT - 7

7.Write a Program to implement Sliding window protocol for Selective repeat.

```
#include<stdio.h>
int main()
{
int windowsize,sent=0,ack,i,c;
printf("Enter window size:");
scanf("%d",&windowsize);
while(1)
{
for(i=0;i<windowsize;i++)
{
if(sent>=windowsize)
{
break;
}
printf("Frame %d transmitted\n= , sent);
sent++;
}
printf("\n please enter your choice:\n 1. Enter Acknowledgement\n 2.Exit\n");
scanf("%d",&c);
if(c==1)
{
Int ack1;
printf("Enter the frame number for acknowledgement:");
scanf("%d",&ack);
if(ack<0||ack>=windowsize)
{
printf("Invalid acknowledgement frame number\n");
continue;
}
if(ack<sent)
{
printf("Frame %d is already received \n",ack);
}
else
{
printf("Frame %d is acknowledged \n",ack);
}
if(ack==windowsize-1)
{
printf("All frames have been acknowledged \n");
break;
```

```
    }
    }
    else if(c==2)
    {
    printf("Exiting……\n");
    break;
}

    else
    {
    printf("Invalid choice please enter 1 or 2 \n");
    }
    return 0;
    }
```

OUTPUT:

```
Enter window size : 2
Frame 0 transmitted
Frame 1 transmitted
Please enter your choice :
1.Enter acknowledgement
2.Exit
1.
Enter the frame number for acknowledgemnt : 0
Frame 0 is already received
Please enter your choice:
1.Enter acknowledgement
2.Exit
2.
Exiting……..
```

EXPERIMENT - 8

8.Write a Program to implement Stop and Wait Protocol.

```
#include<stdio.h>
int main()
{
int size,sent=0,ack,I;
printf("Enter number of freames to be transferred");
scanf("%d",&size);
while(sent<size)
{
printf("frame %d has been transmitted \n",sent);
while(1)
{
printf("Enter your choice:\n 1.For ACK \n 2.For NACK\n");
printf("Choice:");
scanf("%d",&ack);
if(ack==1)
{
printf("The frame %d has been acknowledged \n ",sent);
break;
}
else if(ack ==2)
{
printf("The frame %d has not been acknowledged re-trasmitting……..\n",sent);
continue;
}
else
{
printf("Invalid choice . please enter your choice:\n 1.For ACK \n 2.For NACK\n");
}
}
sent++;
}
printf("All frames have been transmitted and acknowledged.\n");
return 0;
}
```

OUTPUT:

```
Enter no.of frames to be transferred:1
Frame 0 has been transmitted
Enter your choice :
1.For ACK
```

2.For NACK
Choice:
The frame 0 has not been acknowledged & transmitted
Enter your choice:
1.For ACK
2.For NACK
Choice 1;
The frame 0 has been acknowledged
All frames have been transmitted & acknowledged

EXPERIMENT - 9

9.Write a program for congestion control using leaky bucket algorithm

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>

#define NOF_PACKETS 10

int rand(int a)
{
   int rn = (random() % 10) % a;
   return  rn == 0 ? 1 : rn;
}

int main()
{
   int packet_sz[NOF_PACKETS], i, clk, b_size, o_rate, p_sz_rm=0, p_sz, p_time, op;
   for(i = 0; i<NOF_PACKETS; ++i)
      packet_sz[i] = rand(6) * 10;
   for(i = 0; i<NOF_PACKETS; ++i)
      printf("\npacket[%d]:%d bytes\t", i, packet_sz[i]);
   printf("\nEnter the Output rate:");
   scanf("%d", &o_rate);
   printf("Enter the Bucket Size:");
   scanf("%d", &b_size);
   for(i = 0; i<NOF_PACKETS; ++i)
   {
      if( (packet_sz[i] + p_sz_rm) > b_size)
         if(packet_sz[i] > b_size)/*compare the packet siz with bucket size*/
            printf("\n\nIncoming packet size (%dbytes) is Greater than bucket capacity (%dbytes)-
PACKET REJECTED", packet_sz[i], b_size);
         else
            printf("\n\nBucket capacity exceeded-PACKETS REJECTED!!");
      else
      {
         p_sz_rm += packet_sz[i];
         printf("\n\nIncoming Packet size: %d", packet_sz[i]);
         printf("\nBytes remaining to Transmit: %d", p_sz_rm);
         p_time = rand(4) * 10;
```

```
        printf("\nTime left for transmission: %d units", p_time);
        for(clk = 10; clk <= p_time; clk += 10)
        {
          sleep(1);
          if(p_sz_rm)
          {
            if(p_sz_rm <= o_rate)/*packet size remaining comparing with output rate*/
              op = p_sz_rm, p_sz_rm = 0;
            else
              op = o_rate, p_sz_rm -= o_rate;
            printf("\nPacket of size %d Transmitted", op);
            printf("----Bytes Remaining to Transmit: %d", p_sz_rm);
          }
          else
          {
            printf("\nTime left for transmission: %d units", p_time-clk);
            printf("\nNo packets to transmit!!");
          }
        }
      }
    }
}
```

Output:

```
fsmk@fsmk-ThinkCentre-M71e:~/network_done/leaky_bucket$ gcc leakybucket.c
fsmk@fsmk-ThinkCentre-M71e:~/network_done/leaky_bucket$ ./a.out
packet[0]:30 bytes
packet[1]:10 bytes
packet[2]:10 bytes
packet[3]:50 bytes
packet[4]:30 bytes
packet[5]:50 bytes
packet[6]:10 bytes
packet[7]:20 bytes
packet[8]:30 bytes
packet[9]:10 bytes

Enter the Output rate:10
Enter the Bucket Size:15


Incomming packet size (30bytes) is Greater than bucket capacity (15bytes)-PACKET
 REJECTED

Incomming Packet size: 10
Bytes remaining to Transmit: 10
Time left for transmission: 20 units
  Packet of size 10 Transmitted----Bytes Remaining to Transmit: 0
Time left for transmission: 0 units
  No packets to transmit!!
```

```
Incomming Packet size: 10
Bytes remaining to Transmit: 10
Time left for transmission: 30 units
  Packet of size 10 Transmitted----Bytes Remaining to Transmit: 0
Time left for transmission: 10 units
  No packets to transmit!!
Time left for transmission: 0 units
  No packets to transmit!!

Incomming packet size (50bytes) is Greater than bucket capacity (15bytes)-PACKET
 REJECTED

Incomming packet size (30bytes) is Greater than bucket capacity (15bytes)-PACKET
 REJECTED

Incomming packet size (50bytes) is Greater than bucket capacity (15bytes)-PACKET
 REJECTED

Incomming Packet size: 10
Bytes remaining to Transmit: 10
Time left for transmission: 10 units
  Packet of size 10 Transmitted----Bytes Remaining to Transmit: 0

Incomming packet size (20bytes) is Greater than bucket capacity (15bytes)-PACKET
 REJECTED

Incomming packet size (30bytes) is Greater than bucket capacity (15bytes)-PACKET
```

```
Time left for transmission: 0 units
  No packets to transmit!!

Incomming packet size (50bytes) is Greater than bucket capacity (15bytes)-PACKET
 REJECTED

Incomming packet size (30bytes) is Greater than bucket capacity (15bytes)-PACKET
 REJECTED

Incomming packet size (50bytes) is Greater than bucket capacity (15bytes)-PACKET
 REJECTED

Incomming Packet size: 10
Bytes remaining to Transmit: 10
Time left for transmission: 10 units
  Packet of size 10 Transmitted----Bytes Remaining to Transmit: 0

Incomming packet size (20bytes) is Greater than bucket capacity (15bytes)-PACKET
 REJECTED

Incomming packet size (30bytes) is Greater than bucket capacity (15bytes)-PACKET
 REJECTED

Incomming Packet size: 10
Bytes remaining to Transmit: 10
Time left for transmission: 10 units
  Packet of size 10 Transmitted----Bytes Remaining to Transmit: 0fsmk@fsmk-ThinkCentre-M71e:~/network_done/leaky_bucket$
```

EXPERIMENT – 10

10. Write a Program to implement Dijkstra's algorithm to compute the Shortest path through a graph.

```
#include <stdio.h>
#include <conio.h>
#define INFINITY 9999
#define MAX 10

void dijkstra(int G[MAX][MAX], int n, int startnode);

int main() {
   int G[MAX][MAX], i, j, n, u;

   printf("Enter number of vertices: ");
   scanf("%d", &n);

   printf("\nEnter the adjacency matrix:\n");
   for (i = 0; i < n; i++) {
      for (j = 0; j < n; j++) {
         scanf("%d", &G[i][j]);
      }
   }

   printf("\nEnter the starting node: ");
   scanf("%d", &u);

   dijkstra(G, n, u);

   return 0;
}

void dijkstra(int G[MAX][MAX], int n, int startnode) {
   int cost[MAX][MAX], distance[MAX], pred[MAX];
   int visited[MAX], count, mindistance, nextnode, i, j;

   // Create the cost matrix
   for (i = 0; i < n; i++) {
      for (j = 0; j < n; j++) {
         if (G[i][j] == 0)
            cost[i][j] = INFINITY;
         else
            cost[i][j] = G[i][j]; // Assuming G contains weights
      }
```

```
}

// Initialize pred[], distance[], and visited[]
for (i = 0; i < n; i++) {
   distance[i] = cost[startnode][i];
   pred[i] = startnode;
   visited[i] = 0;
}

distance[startnode] = 0;
visited[startnode] = 1;
count = 1;

while (count < n - 1) {
   mindistance = INFINITY;

   // Find the next node with minimum distance
   for (i = 0; i < n; i++) {
      if (!visited[i] && distance[i] < mindistance) {
         mindistance = distance[i];
         nextnode = i;
      }
   }

   // Check if a better path exists through next node
   visited[nextnode] = 1;

   for (i = 0; i < n; i++) {
      if (!visited[i]) {
         if (mindistance + cost[nextnode][i] < distance[i]) {
            distance[i] = mindistance + cost[nextnode][i];
            pred[i] = nextnode;
         }
      }
   }

   count++;
}

// Print the path and distance of each node
for (i = 0; i < n; i++) {
   if (i != startnode) {
      printf("\nDistance of node %d = %d", i, distance[i]);
      printf("\nPath = %d", i);
      j = i;
      do {
```

```
            j = pred[j];
            printf("<-%d", j);
        } while (j != startnode);
    }
  }
}
```

OUTPUT:

```
Enter number of vertices: 3

Enter the adjacency matrix:
1
6
3
8
2
9
3
6
5

Enter the starting node: 1

Distance of node 0 = 8
Path = 0<-1
Distance of node 2 = 9
Path = 2<-1
--------------------------------
Process exited after 34.94 seconds with return value 0
Press any key to continue . . .
```

EXPERIMENT - 11

11. Write a Program to implement Distance vector routing algorithm by obtaining routing table at each node (Take an example subnet graph with weights indicating delay between nodes).

```
#include <stdio.h>
struct node {
   unsigned dist[20]; // Distance from the source node
   unsigned from[20]; // Predecessor node
} rt[10];

int main() {
   int costmat[20][20];
   int nodes, i, j, k, count = 0;

   printf("\nEnter the number of nodes: ");
   scanf("%d", &nodes);

   // Enter the cost matrix
   printf("\nEnter the cost matrix:\n");
   for (i = 0; i < nodes; i++) {
      for (j = 0; j < nodes; j++) {
         scanf("%d", &costmat[i][j]);
         costmat[i][i] = 0; // Distance to self is zero
         rt[i].dist[j] = costmat[i][j]; // Initialise the distance equal to cost matrix
         rt[i].from[j] = j; // Initialize predecessor
      }
   }

   do {
      count = 0;
      for (i = 0; i < nodes; i++) { // We choose arbitrary vertex k and we calculate the direct distance
from the node i to k using the cost matrix
         for (j = 0; j < nodes; j++) {
            for (k = 0; k < nodes; k++) {
               if (rt[i].dist[j] > costmat[i][k] + rt[k].dist[j]) {
                  // We calculate them in minimum distance
                  rt[i].dist[j] = costmat[i][k] + rt[k].dist[j];
                  rt[i].from[j] = k;
                  count++;
               }
            }
         }
      }
   } while (count != 0);
```

```
    for (i = 0; i < nodes; i++) {
        printf("\n\nFor router %d\n", i + 1);
        for (j = 0; j < nodes; j++) {
            printf("\t\nNode %d via %d Distance %d", j + 1, rt[i].from[j] + 1, rt[i].dist[j]);
        }
    }

    printf("\n\n");
    return 0;
}
```

OUTPUT:

```
Enter the number of nodes: 3

Enter the cost matrix:
12
6
3
6
7
8
2
1
9


For router 1

Node 1 via 1 Distance 0
Node 2 via 3 Distance 4
Node 3 via 3 Distance 3

For router 2

Node 1 via 1 Distance 6
Node 2 via 2 Distance 0
Node 3 via 3 Distance 8

For router 3

Node 1 via 1 Distance 2
Node 2 via 2 Distance 1
Node 3 via 3 Distance 0


--------------------------------
Process exited after 9.486 seconds with return value 0
Press any key to continue . . .
```

EXPERIMENT - 12

12.Write a Program to implement Broad cast tree by taking subnet of hosts.

```
#include<stdio.h>
#include<conio.h>

int a[20][20], q[20], visited[20], n, i, j, f = 0, r = -1;

// BFS function to traverse the graph starting from vertex `v`
void bfs(int v) {
   // Mark the current vertex as visited
   visited[v] = 1;

   // Add the starting vertex to the queue
   q[++r] = v;

   // Perform BFS traversal
   while (f <= r) {
      v = q[f++]; // Get the front of the queue and increment the front index

      // Explore all adjacent vertices of the dequeued vertex
      for (i = 1; i <= n; i++) {
         // If an adjacent vertex is found and it is not visited
         if (a[v][i] == 1 && !visited[i]) {
            q[++r] = i;    // Add adjacent vertex to the queue
            visited[i] = 1; // Mark it as visited
         }
      }
   }
}

int main() {
   int v;

   // Clear the screen (clrscr() is deprecated in modern compilers, but it's left commented as per your
request)
   // clrscr();

   // Input: number of vertices
   printf("\nEnter the number of vertices:");
   scanf("%d", &n);

   // Initialize the queue and visited array
   for (i = 1; i <= n; i++) {
      q[i] = 0; // Initialize the queue array
```

```
        visited[i] = 0; // Mark all vertices as not visited
    }

    // Input: adjacency matrix for the graph
    printf("\nEnter graph data in matrix form:\n");
      for (i = 1; i <= n; i++) {
for (j = 1; j <= n; j++) {
            scanf("%d", &a[i][j]);
        }
    }

    // Input: starting vertex for BFS traversal
    printf("\nEnter the starting vertex:");
    scanf("%d", &v);

    // Perform BFS starting from vertex `v`
    bfs(v);

    // Output: print the reachable nodes
    printf("\nThe nodes which are reachable are:\n");
    for (i = 1; i <= n; i++) {
        if (visited[i]) {
            printf("%d\t", i);
        }
    }

    // If BFS couldn't reach any nodes
    if (r == -1) {
        printf("\nBFS is not possible.");
    }

    // Wait for a key press (getch() is deprecated but left for compatibility with older compilers)
    getch();
}
```

OUTPUT:

```
Enter the number of vertices:3

Enter graph data in matrix form:
1
3
2
5
3
5
6
7
5

Enter the starting vertex:1

The nodes which are reachable are:
1
--------------------------------
Process exited after 19.87 seconds with return value 0
Press any key to continue . . .
```

EXPERIMENT - 13

Wireshark

What is Wireshark?

Wireshark is an open-source network protocol analysis software program started by Gerald Combs in 1998. A global organization of network specialists and software developers support Wireshark and continue to make updates for new network technologies and encryption methods.

Wireshark is absolutely safe to use. Government agencies, corporations, non-profits, and educational institutions use Wireshark for troubleshooting and teaching purposes. There isn't a better way to learn networking than to look at the traffic under the Wireshark microscope.

There are questions about the legality of Wireshark since it is a powerful packet sniffer. The Light side of the Force says that you should only use Wireshark on networks where you have permission to inspect network packets. Using Wireshark to look at packets without permission is a path to the Dark Side.



How does Wireshark work?

Wireshark is a packet sniffer and analysis tool. It captures network traffic on the local network and stores that data for offline analysis. Wireshark captures network traffic from Ethernet, Bluetooth, Wireless (IEEE.802.11), Token Ring, Frame Relay connections, and more.

Ed. Note: A "packet" is a single message from any network protocol (i.e., TCP, DNS, etc.)

Ed. Note 2: LAN traffic is in broadcast mode, meaning a single computer with Wireshark can see traffic between two other computers. If you want to see traffic to an external site, you need to capture the packets on the local computer.

Wireshark allows you to filter the log either before the capture starts or during analysis, so you can

narrow down and zero into what you are looking for in the network trace. For example, you can set a filter to see TCP traffic between two IP addresses. You can set it only to show you the packets sent from one computer. The filters in Wireshark are one of the primary reasons it became the standard tool for packet analysis.

How to Download Wireshark

Downloading and installing Wireshark is easy. Step one is to check the official Wireshark Download page for the operating system you need. The basic version of Wireshark is free.

Wireshark for Windows

Wireshark comes in two flavors for Windows, 32 bit and 64 bit. Pick the correct version for your OS. The current release is 3.0.3 as of this writing. The installation is simple and shouldn't cause any issues.

Wireshark for Mac

Wireshark is available on Mac as a Homebrew install.
To install Homebrew, you need to run this command at your Terminal prompt:

/usr/bin/ruby -e "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/master/install)"
Once you have the Homebrew system in place, you can access several open-source projects for your Mac. To install Wireshark run this command from the Terminal:

brew install wireshark
Homebrew will download and install Wireshark and any dependencies so it will run correctly.

Wireshark for Linux

Installing Wireshark on Linux can be a little different depending on the Linux distribution. If you aren't running one of the following distros, please double-check the commands.

Ubuntu

From a terminal prompt, run these commands:

1. sudo apt-get install wireshark
2. sudo dpkg-reconfigure wireshark-common
3. sudo adduser $USER wireshark

Those commands download the package, update the package, and add user privileges to run Wireshark.

Red Hat Fedora

From a terminal prompt, run these commands:

1. sudo dnf install wireshark-qt
2. sudo usermod -a -G wireshark username

The first command installs the GUI and CLI version of Wireshark, and the second adds permissions to use Wireshark.

Kali Linux

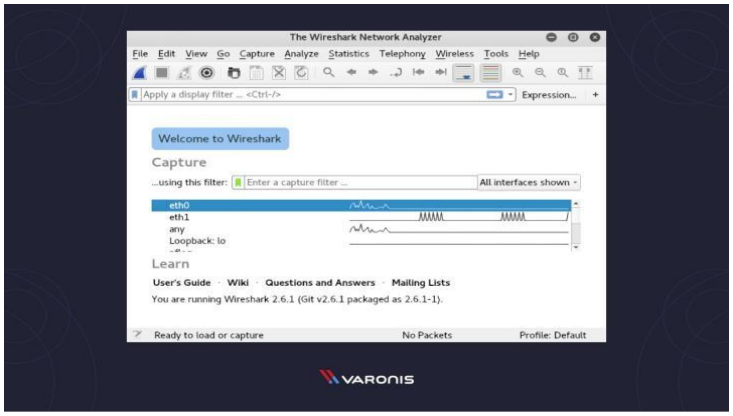
Wireshark is probably already installed! It's part of the basic package. Check your menu to verify. It's under the menu option "Sniffing & Spoofing."

Data Packets on Wireshark

Now that we have Wireshark installed let's go over how to enable the Wireshark packet sniffer and then analyze the network traffic.

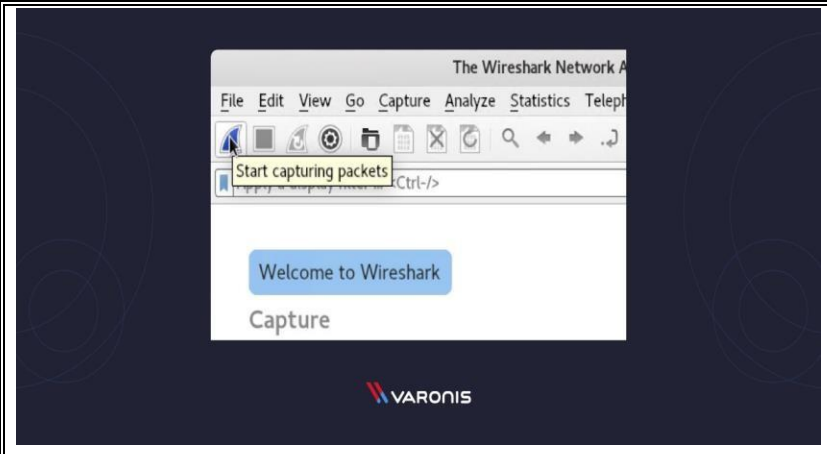
i) Capturing Data Packets on Wireshark

When you open Wireshark, you see a screen that shows you a list of all of the network connections you can monitor. You also have a capture filter field, so you only capture the network traffic you want to see.



You can select one or more of the network interfaces using "shift left -click." Once you have the network interface selected, you can start the capture, and there are several ways to do that.

Click the first button on the toolbar, titled "Start Capturing Packets."



You can select the menu item Capture -> Start.



Or you could use the keystroke Control – E.

During the capture, Wireshark will show you the packets that it captures in real-time.

Once you have captured all the packets you need, you use the same buttons or menu options to stop the capture.

Best practice says that you should stop Wireshark packet capture before you do analysis.

iv) Analyzing Data Packets on Wireshark

Wireshark shows you three different panes for inspecting packet data. The Packet List, the top pane, is a list of all the packets in the capture. When you click on a packet, the other two panes change to show you the details about the selected packet. You can also tell if the packet is part of a conversation. Here are some details about each column in the top pane:

- No.: This is the number order of the packet that got captured. The bracket indicates that this packet is part of a conversation.
- Time: This column shows you how long after you started the capture that this

packet got captured. You can change this value in the Settings menu if you need something different displayed.

- Source: This is the address of the system that sent the packet.
- Destination: This is the address of the destination of that packet.
- Protocol: This is the type of packet, for example, TCP, DNS, DHCPv6, or ARP.
- Length: This column shows you the length of the packet in bytes.
- Info: This column shows you more information about the packet contents, and will vary depending on what kind of packet it is.

Packet Details, the middle pane, shows you as much readable information about the packet as possible, depending on what kind of packet it is. You can right-click and create filters based on the highlighted text in this field.

The bottom pane, Packet Bytes, displays the packet exactly as it got captured in hexadecimal.

When you are looking at a packet that is part of a conversation, you can right -click the packet and select Follow to see only the packets that are part of that conversation.

Wireshark Filters

One of the best features of Wireshark is the Wireshark Capture Filters and Wireshark Display Filters. Filters allow you to view the capture the way you need to see it so you can troubleshoot the issues at hand. Here are several filters to get you started.

Wireshark Capture Filters

Capture filters limit the captured packets by the filter. Meaning if the packets don't match the filter, Wireshark won't save them. Here are some examples of capture filters: host IP-address: this filter limits the capture to traffic to and from the IP address net 192.168.0.0/24: this filter captures all traffic on the subnet.

dst host IP-address: capture packets sent to the specified host. port 53: capture traffic on port 53 only.
port not 53 and not arp: capture all traffic except DNS and ARP traffic

Wireshark Display Filters

Wireshark Display Filters change the view of the capture during analysis. After you have stopped the packet capture, you use display filters to narrow down the packets in the Packet List so you can troubleshoot your issue.

The most useful (in my experience) display filter is:

ip.src==IP-address and ip.dst==IP-address

This filter shows you packets from one computer (ip.src) to another (ip.dst). You can also use ip.addr to show you packets to and from that IP. Here are some others:

tcp.port eq 25: This filter will show you all traffic on port 25, which is usually SMTP traffic.

icmp: This filter will show you only ICMP traffic in the capture, most likely they are pings.

ip.addr != IP_address: This filter shows you all traffic except the traffic to or from the specified computer.
Analysts even build filters to detect specific attacks, like this filter to detect the Sasser worm: ls_ads.opnum==0x09

Additional Wireshark Features

Beyond the capture and filtering, there are several other features in Wireshark that can make your life better.

Wireshark Colorization Options

You can setup Wireshark so it colors your packets in the Packet List according to the display filter, which allows you to emphasize the packets you want to highlight. Check out some examples here.



Wireshark Promiscuous Mode

By default, Wireshark only captures packets going to and from the computer where it runs. By checking the box to run Wireshark in Promiscuous Mode in the Capture Settings, you can capture most of the traffic on the LAN.

Wireshark Command Line

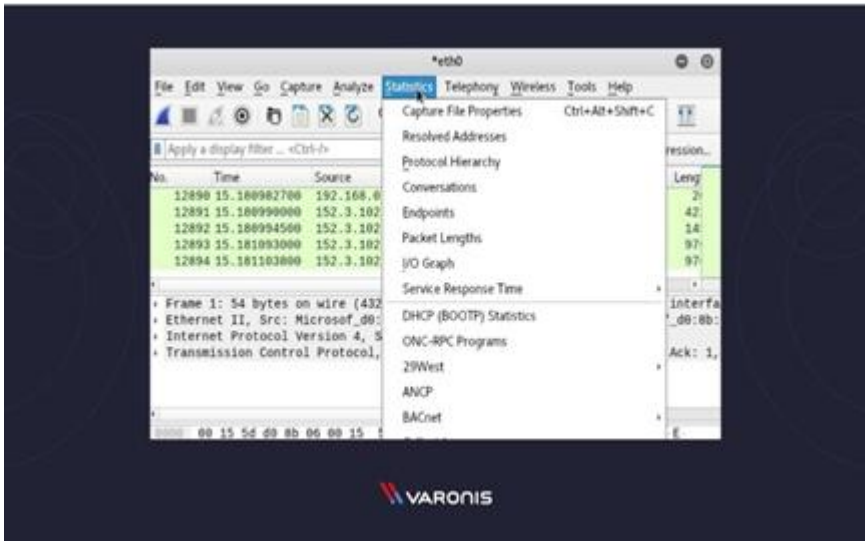
Wireshark does provide a Command Line Interface (CLI) if you operate a system without a GUI. Best practice would be to use the CLI to capture and save a log so you can review the log with the GUI.

Wireshark Commands

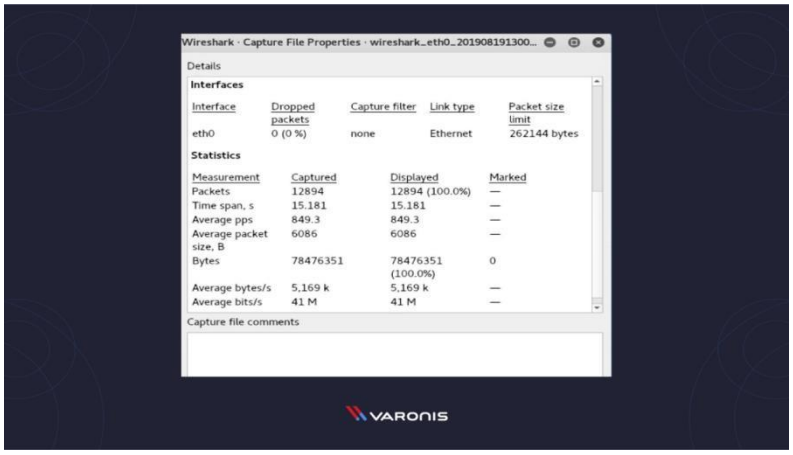
- wireshark : run Wireshark in GUI mode
- wireshark –h : show available command line parameters for Wireshark
- wireshark –a duration:300 –i eth1 –w wireshark. : capture traffic on the Ethernet interface 1 for 5 minutes. –a means automatically stop the capture, -i specifics which interface to capture

Metrics and Statistics

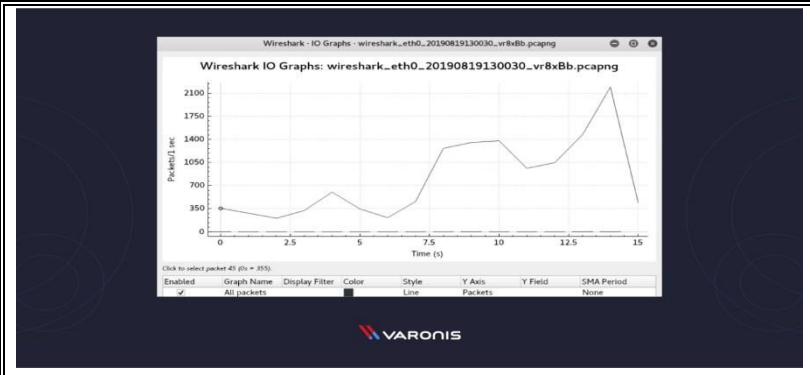
Under the Statistics menu item, you will find a plethora of options to show details about your capture.



Capture File Properties:



Wireshark I/O Graph:



i. Packet Capture Using Wireshark
ii. Starting Wireshark
iii. Viewing Captured Traffic
iv. Analysis and Statistics &Filters.

1. Use Wireshark to perform a packet capture of network traffic

In order to investigate your issues further, we would like to run an analysis of the traffic being sent between you and Salesforce. To do this, we will use the Wireshark application. Wireshark is a tool that allows packet traces to be monitored, captured and analysed. Please follow the steps below in order to obtain a capture of your network traffic using Wireshark.

Note: You will require some administrator rights on your machine in order to complete these tests. If you are unsure, please contact your IT administrator.

Installing the Wireshark package

Visit the Wireshark download site, and download the appropriate Wireshark package or installer for the operating system running on the system which is to be used for packet capture.

When installing, ensure all components are selected for installation, including the optional "Winpcap" application.

Once complete, start Wireshark via shortcut or start menu.
Capturing your traffic with Wireshark
After starting Wireshark, do the following:

1. Select **Capture | Interfaces**
2. Select the interface on which packets need to be captured. This will usually be the interface where the Packet/s column is constantly changing, which would indicate the presence of live traffic). If you have multiple network interface cards (i.e. LAN card and Wi-Fi adapter) you may need to check with your IT administrator to determine the right interface.
3. Click the **Start** button to start the capture.
4. Recreate the problem. The capture dialog should show the number of packets increasing. Try to avoid running any other internet applications while capturing, closing other browsers, Instant

messengers etc.

5. Once the problem which is to be analyzed has been reproduced, click on Stop. It may take a few seconds for Wireshark to display the packets captured.

6. Save the packet trace in the default format. Click on the File menu option and select Save As. By default Wireshark will save the packet trace in libpcap format. This is a filename with a.pcap extension.

Returning the information to Salesforce support

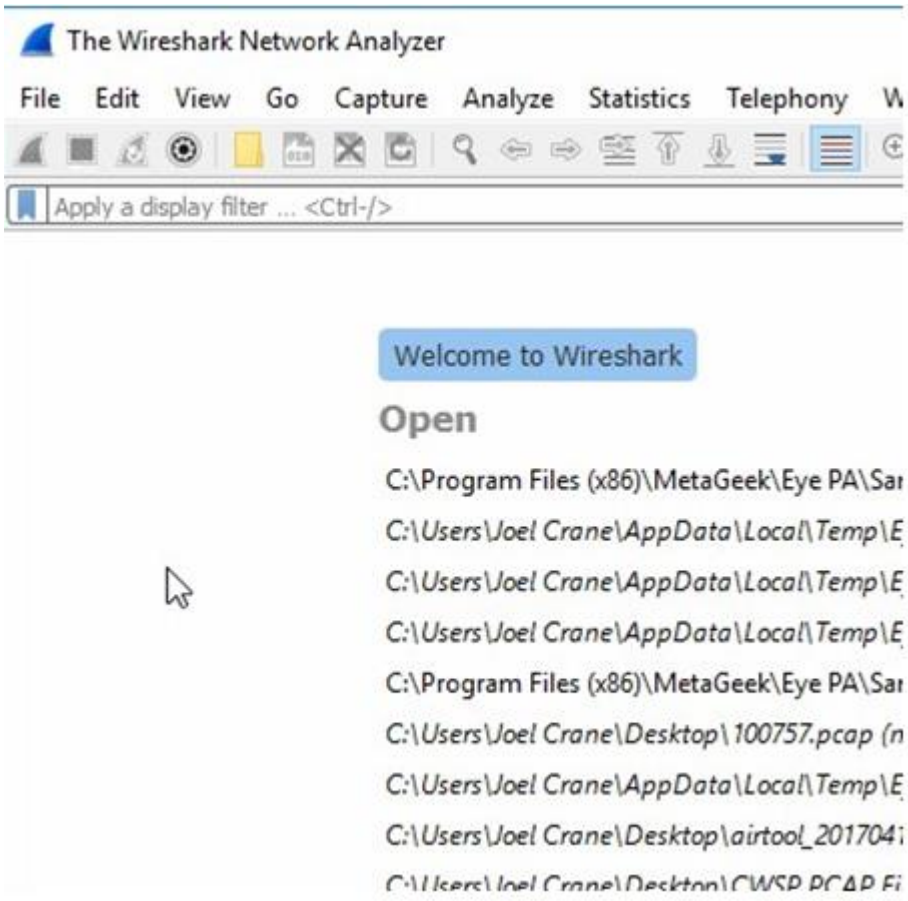
Forward the resulting .pcap file to your support representative, either by email, or attaching it to your open case. Please also include the following information:

- Your external IP address (get this from http://www.whatismyip.com/)
- The internal IP address of the local machine where traffic is being captured
- A click path of the steps you took to reproduce, including links to each page/record accessed

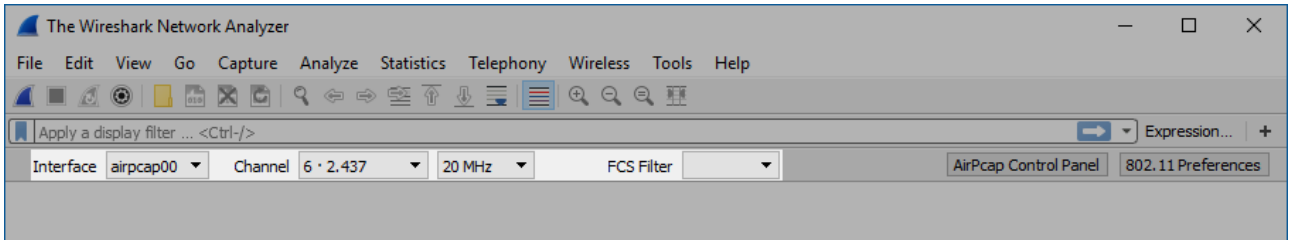
In some cases, you may want to perform packet captures with Wireshark. One case might be when you want to perform a packet capture on channel 12 or 13.

Set up the Packet Capture

1. Click View > Wireless Toolbar. The Wireless Toolbar will appear just below the Main toolbar.



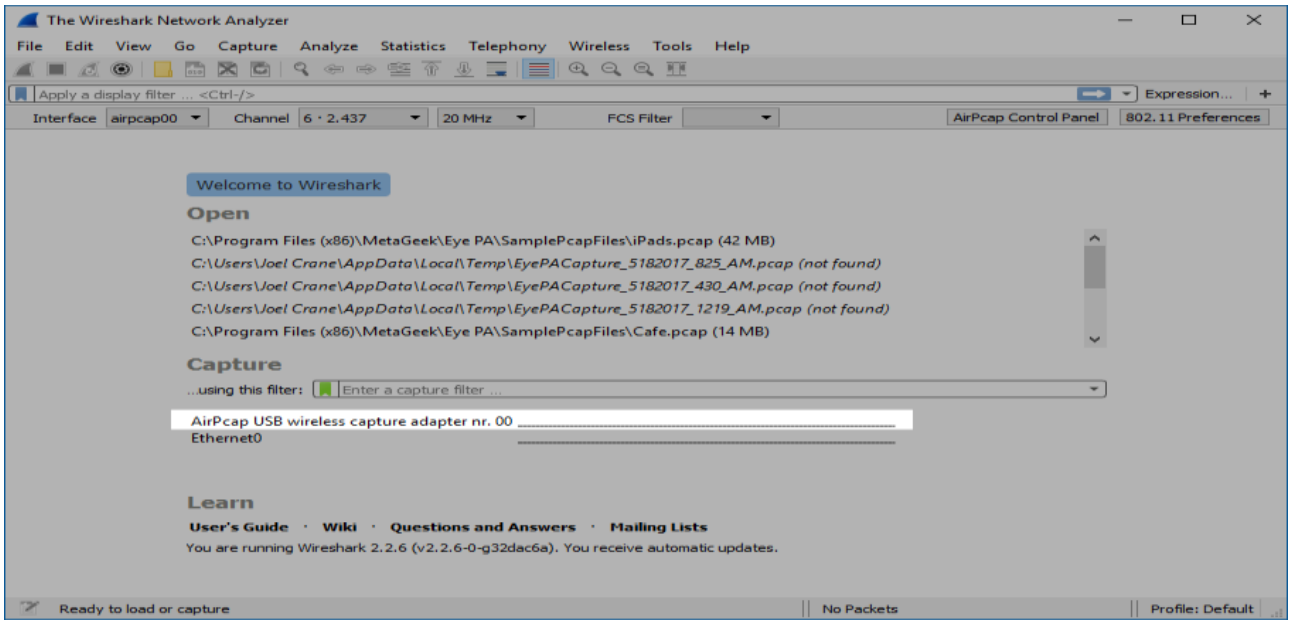
2. Use the **Wireless Toolbar** to configure the desired channel and channel width.



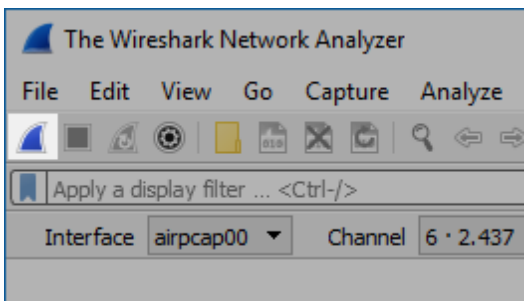
3. Under Capture, click on AirPcap USB wireless capture adapter to select the capture interface.

Note: If the AirPcap isn't listed, press F5 to refresh the list of available packet capture interfaces.

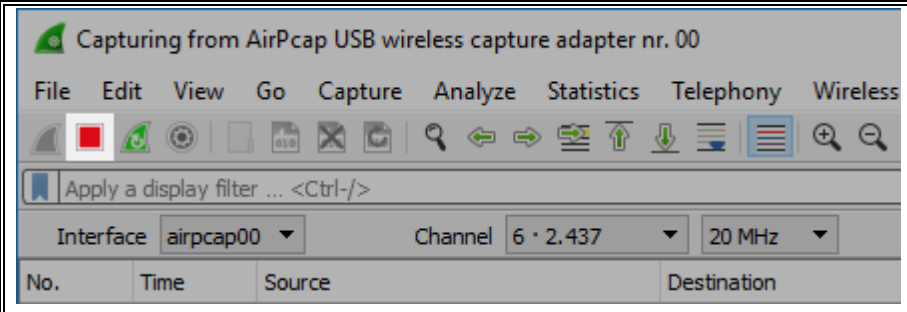
Note: The AirPcap has been discontinued by RiverBed and is 802.11n only.



4. Click the **Start Capture** button to begin the capture.

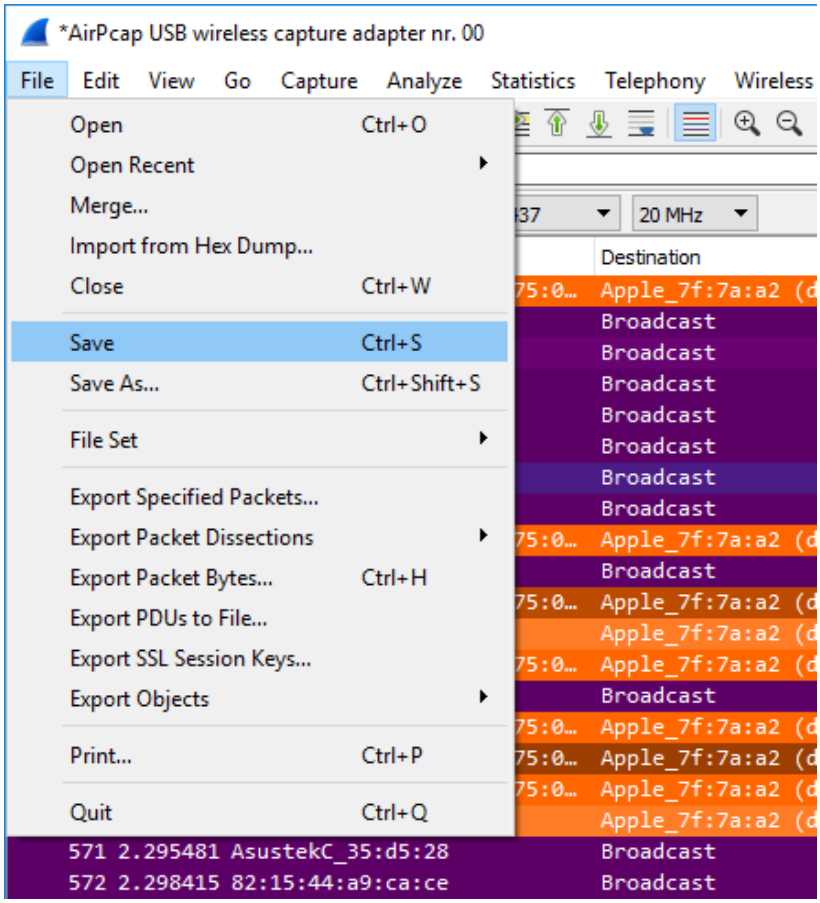


5. When you are finished capturing, click the **Stop** button.



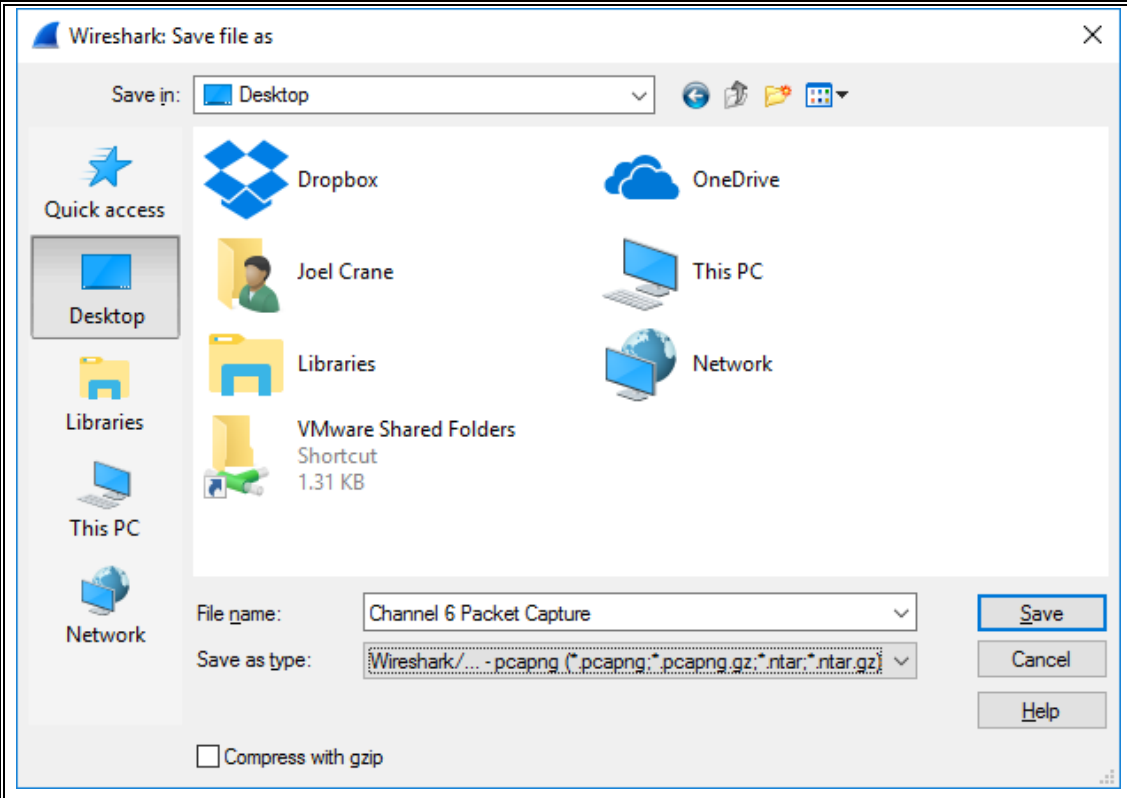
Saving the Capture

1. To save the capture, click **File** > **Save**.



2. Name the file, and click **Save**.

*Note: **.Pcap** and **.Pcap-ng** are good filetypes to use for the capture if you plan to use Eye P.A. to open the capture.*

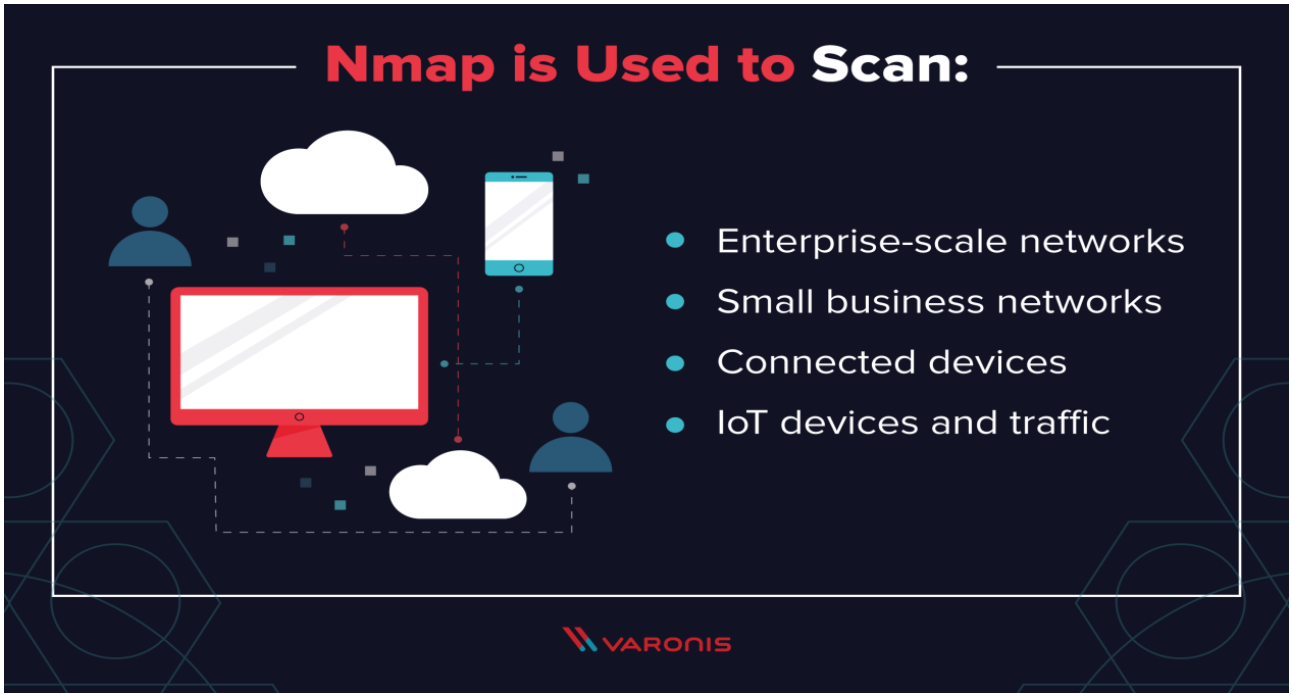


4. **Eye P.A. can now open the capture file.**

EXPERIMENT - 14

14. How to run Nmapscan

What is Nmap?



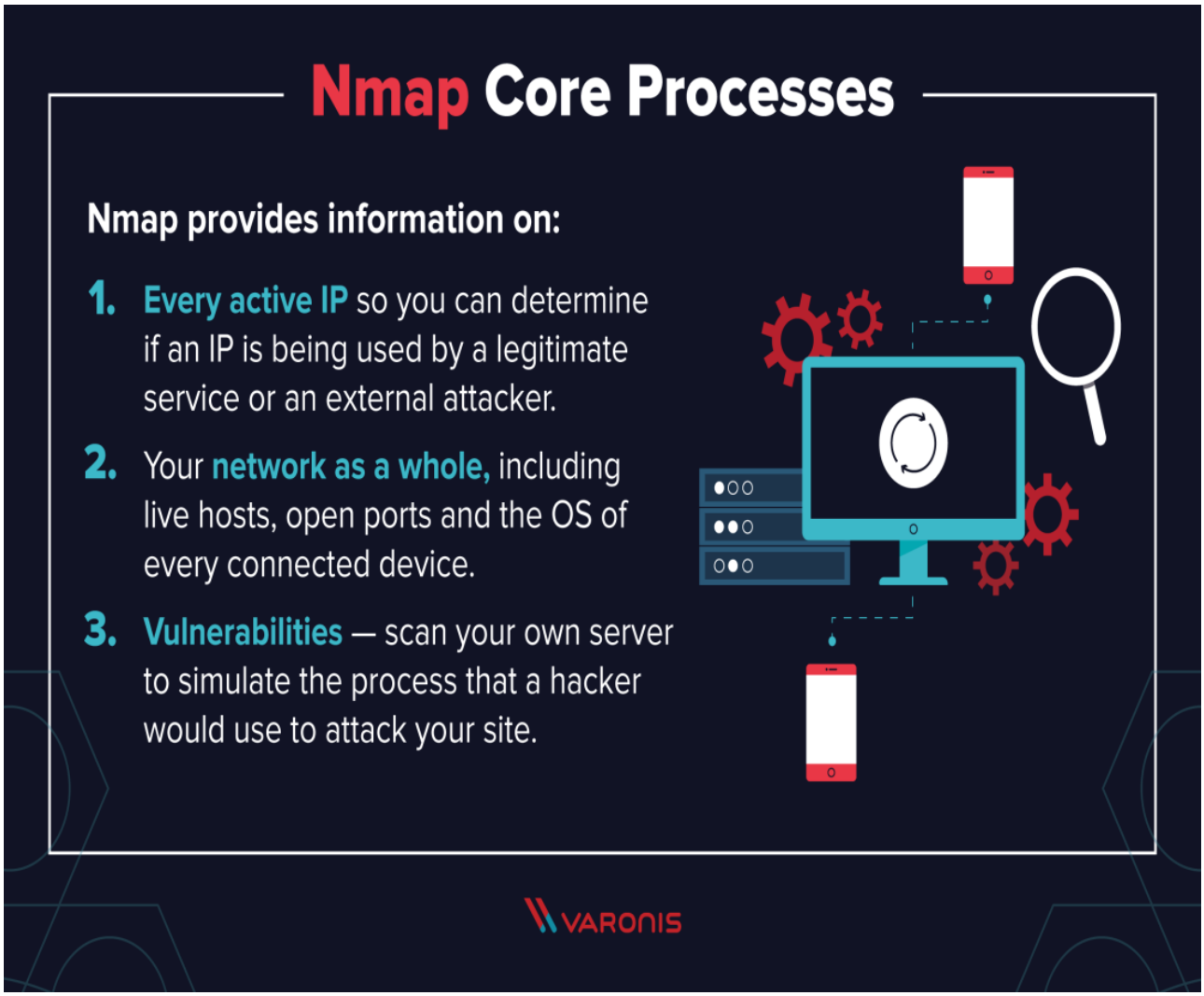
At its core, Nmap is a network scanning tool that uses IP packets to identify all the devices connected to a network and to provide information on the services and operating systems they are running.

The program is most commonly used via a command-line interface (though GUI front-ends are also available) and is available for many different operating systems such as Linux, Free BSD, and Gentoo. Its popularity has also been bolstered by an active and enthusiastic user support community.

Nmap was developed for enterprise-scale networks and can scan through thousands of connected devices. However, in recent years Nmap is being increasingly used by smaller companies. The rise of the IoT, in particular, now means that the networks used by these companies have become more complex and therefore harder to secure.

This means that Nmap is now used in many website monitoring tools to audit the traffic between web servers and IoT devices. The recent emergence of IoT botnets, like Mirai, has also stimulated interest in Nmap, not least because of its ability to interrogate devices connected via the UPnP protocol and to highlight any devices that may be malicious.

What Does Nmap Do?



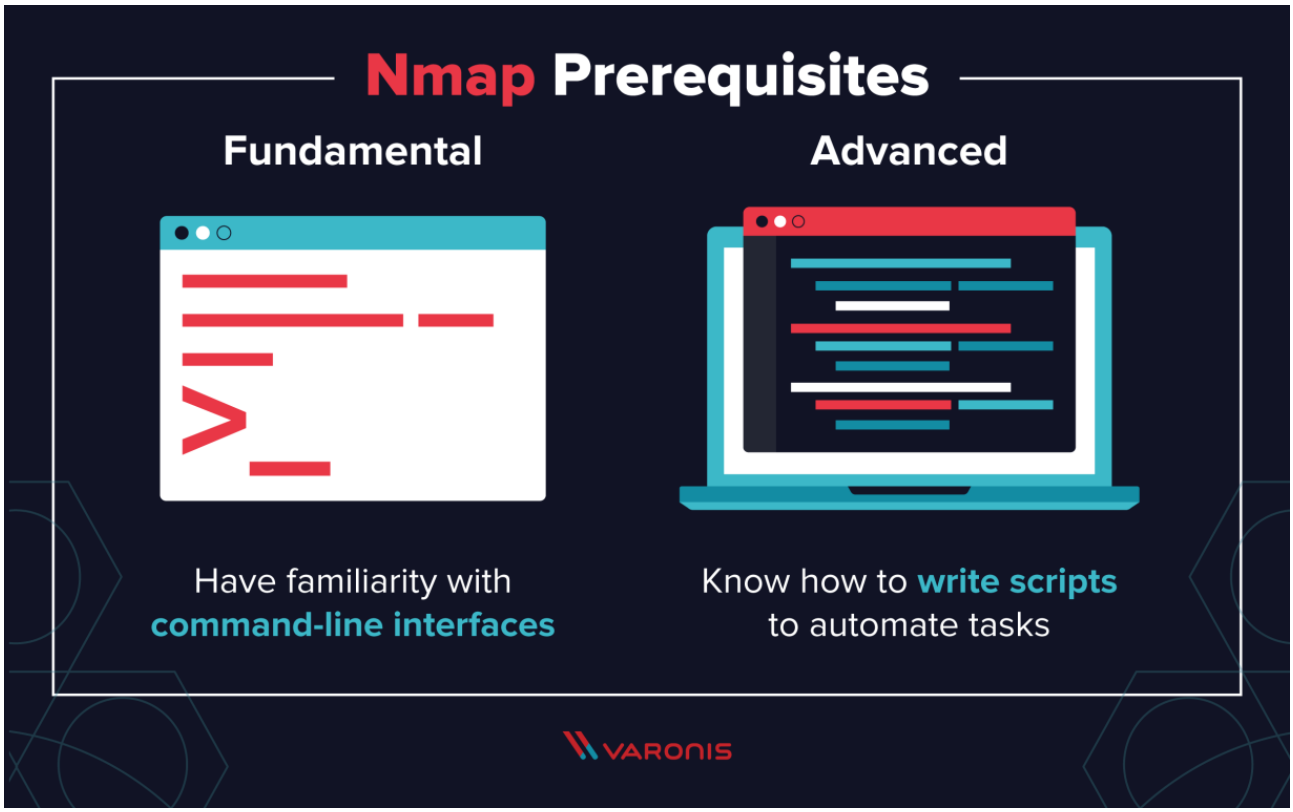
At a practical level, Nmap is used to provide detailed, real-time information on your networks, and on the devices connected to them.

The primary uses of Nmap can be broken into three core processes. First, the program gives you detailed information on every IP active on your networks, and each IP can then be scanned. This allows administrators to check whether an IP is being used by a legitimate service, or by an external attacker.

Secondly, Nmap provides information on your network as a whole. It can be used to provide a list of live hosts and open ports, as well as identifying the OS of every connected device. This makes it a valuable tool in ongoing system monitoring, as well as a critical part of pentesting. Nmap can be used alongside the Metasploit framework, for instance, to probe and then repair network vulnerabilities.

Thirdly, Nmap has also become a valuable tool for users looking to protect personal and business websites. Using Nmap to scan your own web server, particularly if you are hosting your website from home, is essentially simulating the process that a hacker would use to attack your site. "Attacking" your own site in this way is a powerful way of identifying security vulnerabilities.

How To Use Nmap



Nmap is straightforward to use, and most of the tools it provides are familiar to system admins from other programs. The advantage of Nmap is that it brings a wide range of these tools into one program, rather than forcing you to skip between separate and discrete network monitoring tools. In order to use Nmap, you need to be familiar with command-line interfaces. Most advanced users are able to write scripts to automate common tasks, but this is not necessary for basic network monitoring.

How To Install Nmap

The process for installing Nmap is easy but varies according to your operating system. The Windows, Mac, and Linux versions of the program can be downloaded here.

- For Windows, Nmap comes with a custom installer (namp<version>setup.exe). Download and run this installer, and it automatically configures Nmap on your system.
- On Mac, Nmap also comes with a dedicated installer. Run the Nmap-<version>mpkg file to start this installer. On some recent versions of macOS, you might see a warning that Nmap is an “unidentified developer”, but you can ignore this warning.
- Linux users can either compile Nmap from source or use their chosen package manager. To use apt, for instance, you can run Nmap –version to check if Nmap is installed, and sudo apt-get install Nmap to install it.

Nmap Tutorial and Examples

Once you’ve installed Nmap, the best way of learning how to use it is to perform some basic network scans.

How To Run a Ping Scan

One of the most basic functions of Nmap is to identify active hosts on your network. Nmap does this by using a ping scan. This identifies all of the IP addresses that are currently online without sending any packers to these hosts.

To run a ping scan, run the following command:

1. # nmap -sp 192.100.1.1/24

This command then returns a list of hosts on your network and the total number of assigned IP addresses. If you spot any hosts or IP addresses on this list that you cannot account for, you can then run further commands (see below) to investigate them further.

How To Run A Host Scan

A more powerful way to scan your networks is to use Nmap to perform a host scan. Unlike a ping scan, a host scan actively sends ARP request packets to all the hosts connected to your network. Each host then responds to this packet with another ARP packet containing its status and MAC address.

To run a host scan, use the following command:

1. # nmap -sp <target IP range>

This returns information on every host, their latency, their MAC address, and also any description associated with this address. This can be a powerful way of spotting suspicious hosts connected to your network.

If you see anything unusual in this list, you can then run a DNS query on a specific host, by using:

1. # namp -sL <IP address>

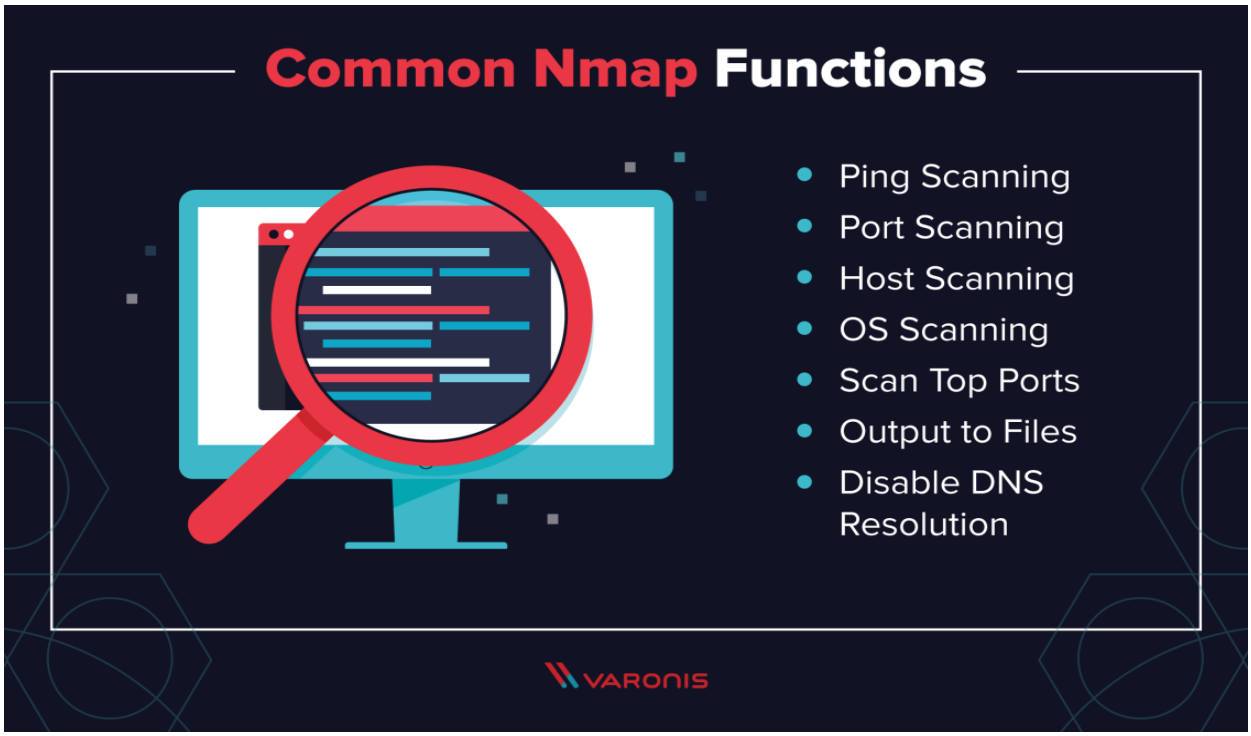
This returns a list of names associated with the scanned IP. This description provides information on what the IP is actually for.

How To Use Nmap in Kali Linux

Using Nmap in Kali Linux can be done in an identical way to running the program on any other flavor of Linux.

That said, there are advantages to using Kali when running Nmap scans. Most modern distros of Kali now come with a fully-features Nmap suite, which includes an advanced GUI and results viewer (Zenmap), a flexible data transfer, redirection, and debugging tool (Ncat), a utility for comparing scan results (Ndiff), and a packet generation and response analysis tool (Nping).

Nmap Commands



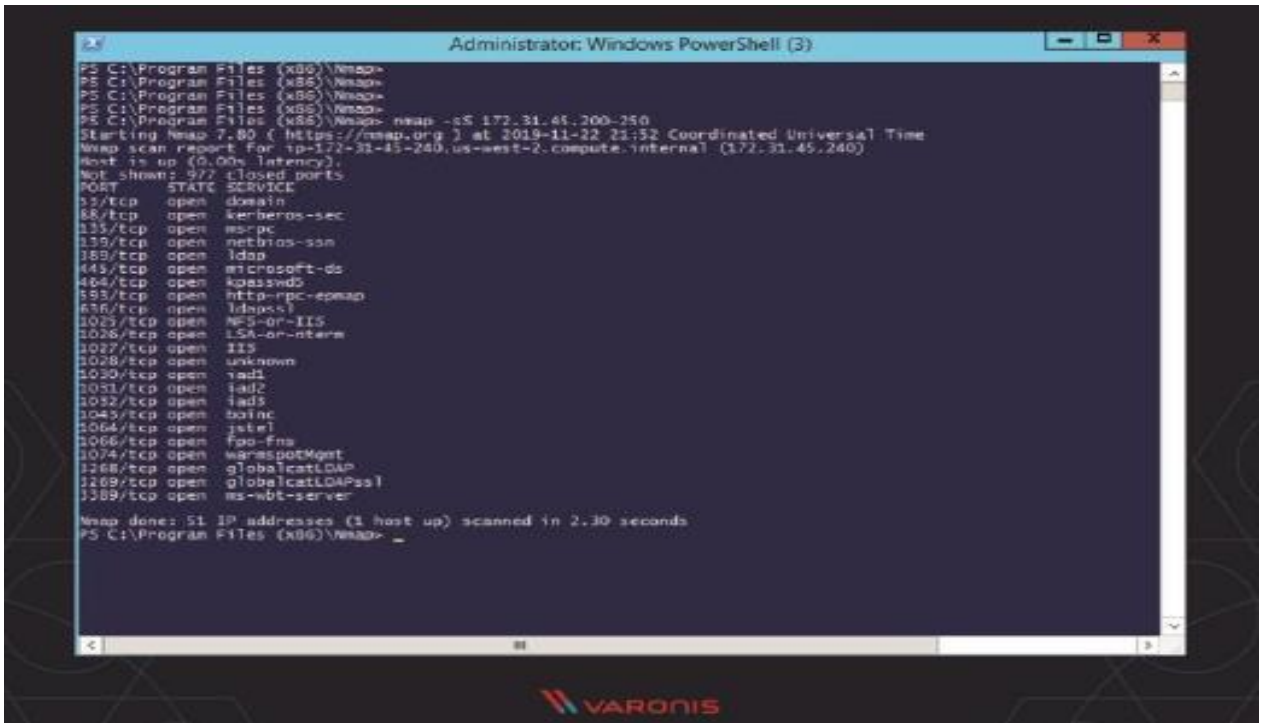
Most of the common functions of Nmap can be executed using a single command, and the program also uses a number of 'shortcut' commands that can be used to automate common tasks.

Here is a quick run-down:

1. Ping Scanning

As mentioned above, a ping scan returns information on every active IP on your network. You can execute a ping scan using this command:

2. Port Scanning



There are several ways to execute port scanning using Nmap. The most commonly used are these:

1. # sS TCP SYN scan
2.
3. # sT TCP connect scan
4.
5. # sU UDP scans
6.
7. # sY SCTP INIT scan
8.
9. # sN TCP NULL

The major differences between these types of scans are whether they cover TCP or UDP ports and whether they execute a TCP connection. Here are the basic differences:

- The most basic of these scans is the sS TCP SYN scan, and this gives most users all the information they need. It scans thousands of ports per second, and because it doesn't complete a TCP connection it does not arouse suspicion.
- The main alternative to this type of scan is the TCP Connect scan, which actively queries each host, and requests a response. This type of scan takes longer than a SYN scan, but can return more reliable information.
- The UDP scan works in a similar way to the TCP connect scan but uses UDP packets to scan DNS, SNMP, and DHCP ports. These are the ports most frequently targeted by hackers, and so this type of scan is a useful tool for checking for vulnerabilities.
- The SCTP INIT scan covers a different set of services: SS7 and SIGTRAN. This type of scan can also be used to avoid suspicion when scanning an external network because it doesn't complete the full SCTP process.
- The TOP NULL scan is also a very crafty scanning technique. It uses a loophole in the TCP system that can reveal the status of ports without directly querying them, which means that you can see their status even where they are protected by a firewall.

3. Host Scanning

Host scanning returns more detailed information on a particular host or a range of IP addresses. As mentioned above, you can perform a host scan using the following command:

1. # nmap -sp <target IP range>

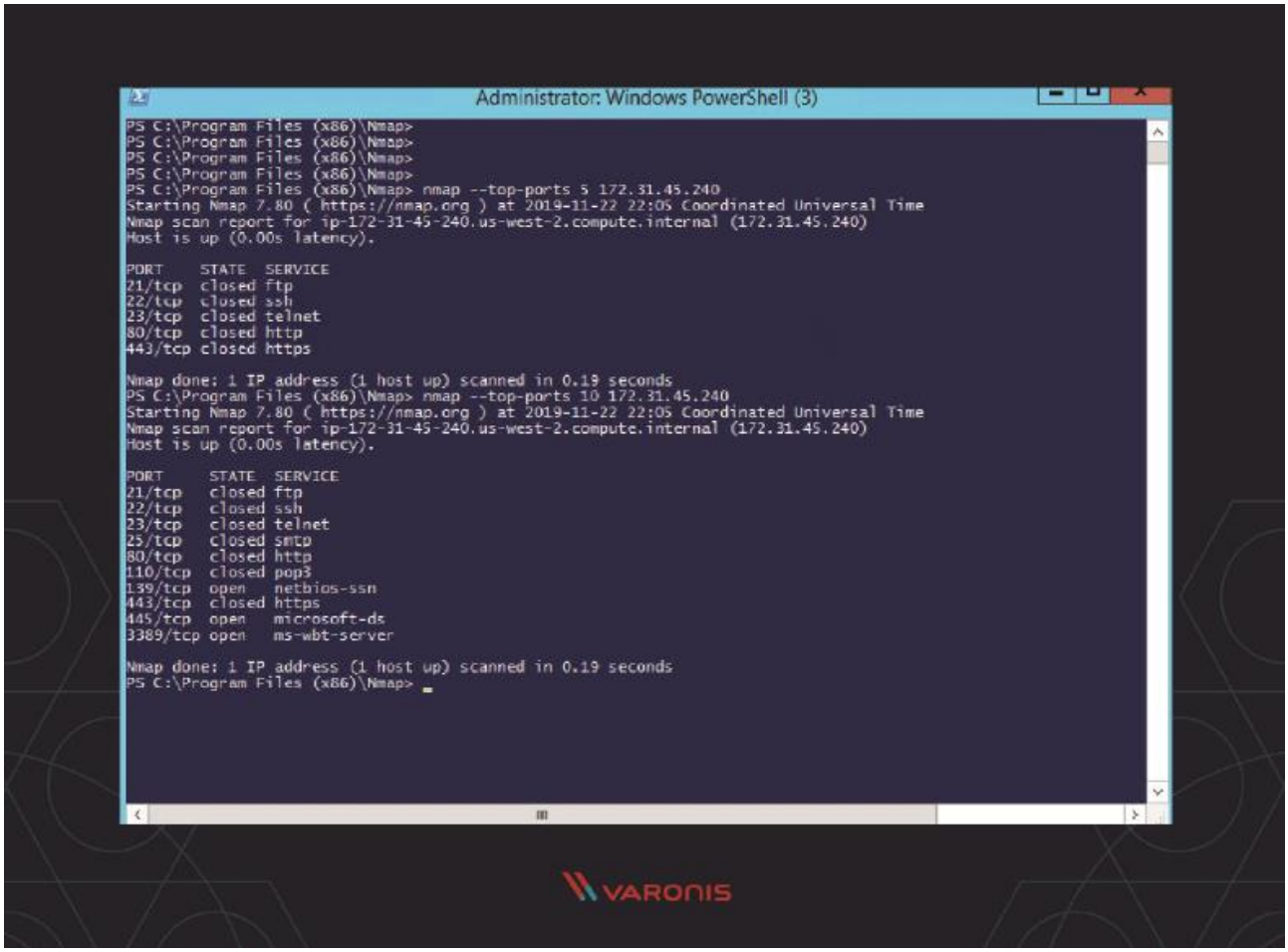
4. OS Scanning

OS scanning is one of the most powerful features of Nmap. When using this type of scan, Nmap sends TCP and UDP packets to a particular port, and then analyze its response. It compares this response to a database of 2600 operating systems, and return information on the OS (and version) of a host.

To run an OS scan, use the following command:

1. nmap -O <target IP>

5. Scan The Most Popular Ports



If you are running Nmap on a home server, this command is very useful. It automatically scans a number of the most 'popular' ports for a host. You can run this command using:

1. nmtop-ports 20 192.168.1.106

Replace the "20" with the number of ports to scan, and Nmap quickly scans that many ports. It returns a concise output that details the status of the most common ports, and this lets you quickly see whether you have any unnecessarily open ports.

6. Output to a File

If you want to output the results of your Nmap scans to a file, you can add an extension to your commands to do that. Simply add:

1. -oN output.txt

To your command to output the results to a text file, or:

1. -oX output.xml

To output to an XML.

7. Disable DNS Name Resolution

Finally, you can speed up your Nmap scans by using the -n parameter to disable reverse DNS resolution. This can be extremely useful if you want to scan a large network. For example, to turn

off DNS resolution for the basic ping scan mentioned above, add -n:

1. # nmap -sp -n 192.100.1.1/24

Nmap FAQ

The commands above cover most of the basic functionality of Nmap. You might still have some questions though, so let's run through the most common ones.

Q: What Are Some Nmap Alternatives?

There are some alternatives to Nmap, but most of them are focused on providing specific, niche functionality that the average system administrator does need frequently. MASSCAN, for instance, is much faster than Nmap but provides less detail. Umit, by contrast, allows you to run several scans at once.

In reality, however, Nmap provides all the functionality and speed that the average user requires, especially when used alongside other similarly popular tools like NetCat (which can be used to manage and control network traffic) and ZenMap (which provides a GUI for Nmap)

Q: How Does Nmap Work?

Nmap builds on previous network auditing tools to provide quick, detailed scans of network traffic. It works by using IP packets to identify the hosts and IPs active on a network and then analyze these packets to provide information on each host and IP, as well as the operating systems they are running.

Q: Is Nmap Legal?

Yes. If used properly, Nmap helps protect your network from hackers, because it allows you to quickly spot any security vulnerabilities in your systems.

Whether port scanning on external servers is legal is another issue. The legislation in this area is complex and varies by territory. Using Nmap to scan external ports can lead to you being banned by your ISP, so make sure you research the legal implications of using the program before you start using it more widely.

EXPERIMENT - 15

15. Operating System Detection using Nmap

OS Detection

One of Nmap's best-known features is remote OS detection using TCP/IP stack fingerprinting. Nmap sends a series of TCP and UDP packets to the remote host and examines practically every bit in the responses. After performing dozens of tests such as TCP ISN sampling, TCP options support and ordering, IP ID sampling, and the initial window size check, Nmap compares the results to its nmap-os-db database of more than 2,600 known OS fingerprints and prints out the OS details if there is a match. Each fingerprint includes a freeform textual description of the OS, and a classification which provides the vendor name (e.g. Sun), underlying OS (e.g. Solaris), OS generation (e.g. 10), and device type (general purpose, router, switch, game console, etc). Most fingerprints also have a Common Platform Enumeration (CPE) representation, like cpe:/o:linux:linux_kernel:2.6.

If Nmap is unable to guess the OS of a machine, and conditions are good (e.g. at least one open port and one closed port were found), Nmap will provide a URL you can use to submit the fingerprint if you know (for sure) the OS running on the machine. By doing this you contribute to the pool of operating systems known to Nmap and thus it will be more accurate for everyone.

OS detection enables some other tests which make use of information that is gathered during the process anyway. One of these is TCP Sequence Predictability Classification. This measures approximately how hard it is to establish a forged TCP connection against the remote host. It is useful for exploiting source-IP based trust relationships (rlogin, firewall filters, etc) or for hiding the source of an attack. This sort of spoofing is rarely performed any more, but many machines are still vulnerable to it. The actual difficulty number is based on statistical sampling and may fluctuate. It is generally better to use the English classification such as “worthy challenge” or “trivial joke”. This is only reported in normal output in verbose (-v) mode. When verbose mode is enabled along with -O, IP ID sequence generation is also reported. Most machines are in the “incremental” class, which means that they increment the ID field in the IP header for each packet they send. This makes them vulnerable to several advanced information gathering and spoofing attacks.

Another bit of extra information enabled by OS detection is a guess at a target's uptime. This uses the TCP timestamp option (RFC 1323) to guess when a machine was last rebooted. The guess can be inaccurate due to the timestamp counter not being initialized to zero or the counter overflowing and wrapping around, so it is printed only in verbose mode.

OS detection is enabled and controlled with the following options:

-O (Enable OS detection)

Enables OS detection, as discussed above. Alternatively, you can use -A to enable OS detection along with other things.

--osscan-limit (Limit OS detection to promising targets)

OS detection is far more effective if at least one open and one closed TCP port are found. Set this option and Nmap will not even try OS detection against hosts that do not meet this criteria. This can save substantial time, particularly on -Pn scans against many hosts. It only matters when OS detection is requested with -O or -A.

--osscan-guess; --fuzzy (Guess OS detection results)

When Nmap is unable to detect a perfect OS match, it sometimes offers up near-matches as possibilities. The match has to be very close for Nmap to do this by default. Either of these (equivalent) options make Nmap guess more aggressively. Nmap will still tell you when an imperfect match is printed and display its confidence level (percentage) for each guess.

--max-os-tries (Set the maximum number of OS detection tries against a target)

When Nmap performs OS detection against a target and fails to find a perfect match, it usually repeats the attempt. By default, Nmap tries five times if conditions are favorable for OS fingerprint submission, and twice when conditions aren't so good. Specifying a lower --max-os-tries value (such as 1) speeds Nmap up, though you miss out on retries which could potentially identify the OS. Alternatively, a high value may be set to allow even more retries when conditions are favorable. This is rarely done, except to generate better fingerprints for submission and integration into the Nmap OS database.

EXPERIMENT - 16

16. Do the following using NS2Simulator

i. NS2Simulator-Introduction
ii. Simulate to Find the Number of Packets Dropped
iii. Simulate to Find the Number of Packets Dropped by TCP/UDP
iv. Simulate to Find the Number of Packets Dropped due to Congestion
v. Simulate to Compare Data Rate& Throughput.
vi. Simulate to Plot Congestion for Different Source/Destination
vii. Simulate to Determine the Performance with respect to Transmission of Packets

SIMULATION USING NS-2

i)Introduction to NS-2:

NS2 is an open-source simulation tool that runs on Linux. It is a discreet event simulator targeted at networking research and provides substantial support for simulation of routing, multicast protocols and IP protocols, such as UDP, TCP, RTP and SRM over wired and wireless (local and satellite) networks.

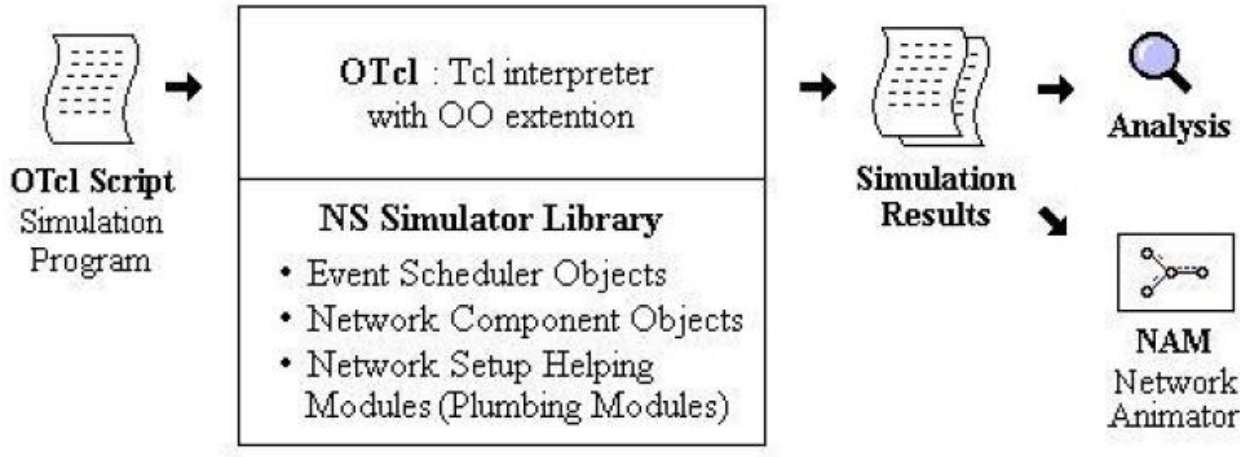
Widely known as NS2, is simply an event driven simulation tool.

Useful in studying the dynamic nature of communication networks.

Simulation of wired as well as wireless network functions and protocols (e.g., routing algorithms, TCP, UDP) can be done using NS2.

In general, NS2 provides users with a way of specifying such network protocols and simulating their corresponding behaviors.

Basic Architecture of NS2



TCL – Tool Command Language

Tcl is a very simple programming language. If you have programmed before, you can learn enough to write interesting Tcl programs within a few hours. This page provides a quick overview of the main features of Tcl. After reading this you'll probably be able to start writing simple Tcl scripts on your own; however, we recommend that you consult one of the many available Tcl books for more

complete information.

Basic syntax

Tcl scripts are made up of *commands* separated by newlines or semicolons.

Commands all have the same basic form illustrated by the following example:

expr 20 + 10

This command computes the sum of 20 and 10 and returns the result, 30. You can try out this example and all the others in this page by typing them to a Tcl application such as tclsh; after a command completes, tclsh prints its result.

Each Tcl command consists of one or more *words* separated by spaces. In this example there are four words: expr, 20, +, and 10. The first word is the name of a command and the other words are *arguments* to that command. All Tcl commands consist of words, but different commands treat their arguments differently. The expr command treats all of its arguments together as an arithmetic expression, computes the result of that expression, and returns the result as a string. In the expr command the division into words isn't significant: you could just as easily have invoked the same command as

expr 20+10

However, for most commands the word structure is important, with each word used for a distinct purpose.

All Tcl commands return results. If a command has no meaningful result then it returns an empty string as its result.

Variables

Tcl allows you to store values in variables and use the values later in commands. The set command is used to write and read variables. For example, the following command modifies the variable x to hold the value 32:

set x 32

The command returns the new value of the variable. You can read the value of a variable by invoking set with only a single argument:

set x

You don't need to declare variables in Tcl: a variable is created automatically the

first time it is set. Tcl variables don't have types: any variable can hold any value.

To use the value of a variable in a command, use *variable substitution* as in the following example:

expr $x*3

When a $ appears in a command, Tcl treats the letters and digits following it as a variable name, and substitutes the value of the variable in place of the name. In this example, the actual argument received by the expr command will be 32*3 (assuming that variable x was set as in the previous example). You can use variable substitution in any word of any command, or even multiple times within a word:

set cmd expr

set x 11

$cmd $x*$x

Command substitution

You can also use the result of one command in an argument to another command.

This is called *command substitution*:

```
set a 44
set b [expr $a*4]
```

When a [appears in a command, Tcl treats everything between it and the matching] as a nested Tcl command. Tcl evaluates the nested command and substitutes its result into the enclosing command in place of the bracketed text. In the example above the second argument of the second set command will be 176.

Quotes and braces

Double-quotes allow you to specify words that contain spaces. For example, consider the following script:

```
set x 24
set y 18
set z "$x + $y is [expr $x + $y]"
```

After these three commands are evaluated variable z will have the value 24 + 18 is 42. Everything between the quotes is passed to the set command as a single word. Note that (a) command and variable substitutions are performed on the text between the quotes, and (b) the quotes themselves are not passed to the command. If the quotes were not present, the set command would have received 6 arguments, which would have caused an error.

Curly braces provide another way of grouping information into words. They are different from quotes in that no substitutions are performed on the text between the curly braces:

```
set z {$x + $y is [expr $x + $y]}
```

This command sets variable z to the value "$x + $y is [expr $x + $y]".

Control structures

Tcl provides a complete set of control structures including commands for conditional execution, looping, and procedures. Tcl control structures are just commands that take Tcl scripts as arguments. The example below creates a Tcl procedure called power, which raises a base to an integer power:

```
proc power {base p} {
set result 1

while {$p > 0} {
set result [expr $result * $base]
set p [expr $p - 1]
}
return $result
}
```

This script consists of a single command, proc. The proc command takes three arguments: the name of a procedure, a list of argument names, and the body of the procedure, which is a Tcl script. Note that everything between the curly brace at the end of the first line and the curly brace on the last line is passed verbatim to proc as a single argument. The proc command creates a new Tcl command named power that takes two arguments. You can then invoke power with commands like the following:

```
power 2 6
power 1.15 5
```

When power is invoked, the procedure body is evaluated. While the body is executing it can access

its arguments as variables: base will hold the first argument and p will hold the second.
The body of the power procedure contains three Tcl commands: set, while, and return. The while command does most of the work of the procedure. It takes two arguments, an expression ($p > 0) and a body, which is another Tcl script. The while command evaluates its expression argument using rules similar to those of the C programming language and if the result is true (nonzero) then it evaluates the body as a Tcl script. It repeats this process over and over until eventually the expression evaluates to false (zero). In this case the body of the while command multiplied the result value by base and then decrements p. When p reaches zero the result contains the desired power of base. The return command causes the procedure to exit with the value of variable result as the procedure's result.

Where do commands come from?
As you have seen, all of the interesting features in Tcl are represented by commands. Statements are commands, expressions are evaluated by executing commands, control structures are commands, and procedures are commands.
Tcl commands are created in three ways. One group of commands is provided by the Tcl interpreter itself. These commands are called *builtin commands*. They include all of the commands you have seen so far and many more (see below). The builtin commands are present in all Tcl applications.
The second group of commands is created using the Tcl extension mechanism. Tcl provides APIs that allow you to create a new command by writing a *command procedure* in C or C++ that implements the command. You then register the command procedure with the Tcl interpreter by telling Tcl the name of the command that the procedure implements. In the future, whenever that particular name is used for a Tcl command, Tcl will call your command procedure to execute the command. The builtin commands are also implemented using this same extension mechanism; their command procedures are simply part of the Tcl library.
When Tcl is used inside an application, the application incorporates its key features into Tcl using the extension mechanism. Thus the set of available Tcl commands varies from application to application. There are also numerous extension packages that can be incorporated into any Tcl application. One of the best known extensions is Tk, which provides powerful facilities for building graphical user interfaces. Other extensions provide object-oriented programming, database access, more graphical capabilities, and a variety of other features. One of Tcl's greatest advantages for building integration applications is the ease with which it can be extended to incorporate new features or communicate with other resources.
The third group of commands consists of procedures created with the proc command, such as the power command created above. Typically, extensions are used for lower-level functions where C programming is convenient, and procedures are used for higher-level functions where it is easier to write in Tcl.

Wired TCL Script Components
Create the event scheduler
Open new files & turn on the tracing
Create the nodes
Setup the links
Configure the traffic type (e.g., TCP, UDP, etc)
Set the time of traffic generation (e.g., CBR, FTP)
Terminate the simulation

NS Simulator Preliminaries.

Initialization and termination aspects of the ns simulator.
Definition of network nodes, links, queues and topology.
Definition of agents and of applications.
The nam visualization tool.
Tracing and random variables.

Features of NS2

NS2 can be employed in most unix systems and windows. Most of the NS2 code is in C++. It uses TCL as its scripting language, Otcl adds object orientation to TCL.NS(version 2) is an object oriented, discrete event driven network simulator that is freely distributed and open source.

- Traffic Models: CBR, VBR, Web etc
- Protocols: TCP, UDP, HTTP, Routing algorithms,MAC etc
- Error Models: Uniform, bursty etc

- Misc: Radio propagation, Mobility models , Energy Models
- Topology Generation tools
- Visualization tools (NAM), Tracing

Structure of NS

● NS is an object oriented discrete event simulator
– Simulator maintains list of events and executes one event after another
– Single thread of control: no locking or race conditions
● Back end is C++ event scheduler
– Protocols mostly
– Fast to run, more control
• Front end is OTCL
Creating scenarios, extensions to C++ protocols
fast to write and change

Platforms

It can be employed in most unix systems(FreeBSD, Linux, Solaris) and Windows.

Source code

Most of NS2 code is in C++

Scripting language

It uses TCL as its scripting language OTcl adds object orientation to TCL.

Protocols implemented in NS2

Transport layer(Traffic Agent) – TCP, UDP
Network layer(Routing agent)
Interface queue – FIFO queue, Drop Tail queue, Priority queue
Logic link contol layer – IEEE 802.2, AR

How to use NS2

Design Simulation – Determine simulation scenario
Build ns-2 script using tcl.

Run simulation

Simulation with NS2

Define objects of simulation.
Connect the objects to each other
Start the source applications. Packets are then created and are transmitted through network.
Exit the simulator after a certain fixed time.

NS programming Structure

- Create the event scheduler
- Turn on tracing
- Create network topology
- Create transport connections
- Generate traffic
- Insert errors

ii& iii) Simulate a three-node point-to-point network with a duplex link between them. Set the queue size and vary the bandwidth and find the number of packets dropped.

STEPS:

Step1: Select the hub icon on the toolbar and drag it onto the working window.
Step2: Select the host icon on the toolbar and drag it onto the working window. Repeat this for another host icon.
Step3: Select the link icon on the toolbar and drag it on the screen from host (node 1) to the hub and again from host(node 2) to the hub. Here the hub acts as node 3 in the point-to-point network. This leads to the creation of the 3-node point-to-point network topology. Save this topology as a .tpl file.
Step4: Double-click on host(node 1), a host dialog box will open up. Click on Node editor and you can see the different layers- interface, ARP, FIFO, MAC, TCPDUMP, Physical layers. Select MAC and then select full-duplex for switches and routers and half duplex for hubs, and in log Statistics, select Number of Drop Packets, Number of Collisions, Throughput of incoming packets and Throughput of outgoing packets. Select FIFO and set the queue size to 50 and press OK. Then click on Add. Another dialog box pops up. Click on the Command box and type the Command according to the following syntax:
stg [-t duration(sec)] [-p port number]HostIPaddr
and click OK.
Step 5: Double-click on host (node 2), and follow the same step as above with only change in command according to the following syntax:
rtg [-t] [-w log] [-p port number]
and click OK.
Step 6: Double click on the link between node 1 and the hub to set the bandwidth to some initial value say, 10 Mbps. Repeat the same for the other node.
Step 7: Click on the E button (Edit Property) present on the toolbar in order to save the changes made to the topology. Now click on the R button (RunSimulation). By doing so a user can

run/pause/continue/stop/abort/disconnect/reconnect/submit a simulation. No simulation settings can be changed in this mode.

Step 8: Now go to Menu->Simulation->Run. Executing this command will submit he current simulation job to one available simulation server managed by the dispatcher. When the simulation server is executing, the user will see the time knot at the bottom of the screen move. The time knot reflects the current virtual time (progress) of the simulation case.

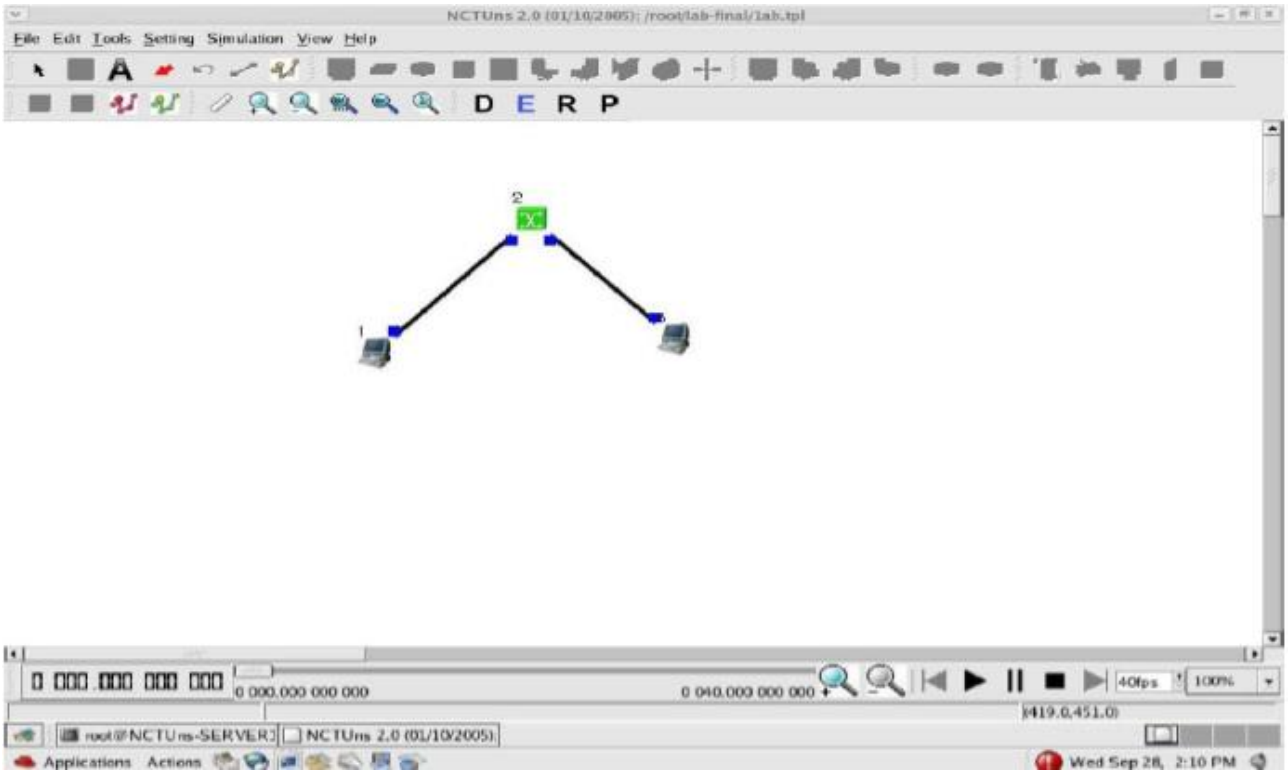
Step 9: To start the playback, the user can left-click the start icon(|>) of the time bar located at the bottom. The animation player will then start playing the recorded packet animation.

Step 10: Change the bandwidth say, 9 Mbps, and run the simulation and compare the two results.

Step 11: To view the results, go to the filename. results folder**.**

Note: To get the syntax of any command, double click on the host icon. Host dialog boxes appear and then choose App. Usage.

The screenshot below explain the topology.



iv)Simulate the transmission of ping messages over a network topology consisting of 6 nodes and find the number of packets dropped due to congestion.

STEPS:

Step 1: Click on the subnet icon on the toolbar and then click on the screen of the working window.

Step 2: Select the required number of hosts and a suitable radius between the host and the switch.

Step 3: In the edit mode, get the IP address of one of the hosts say, host 1 and then for the other host say, host2 set the drop packet and no: of collisions statistics

as described in the earlier experiments.

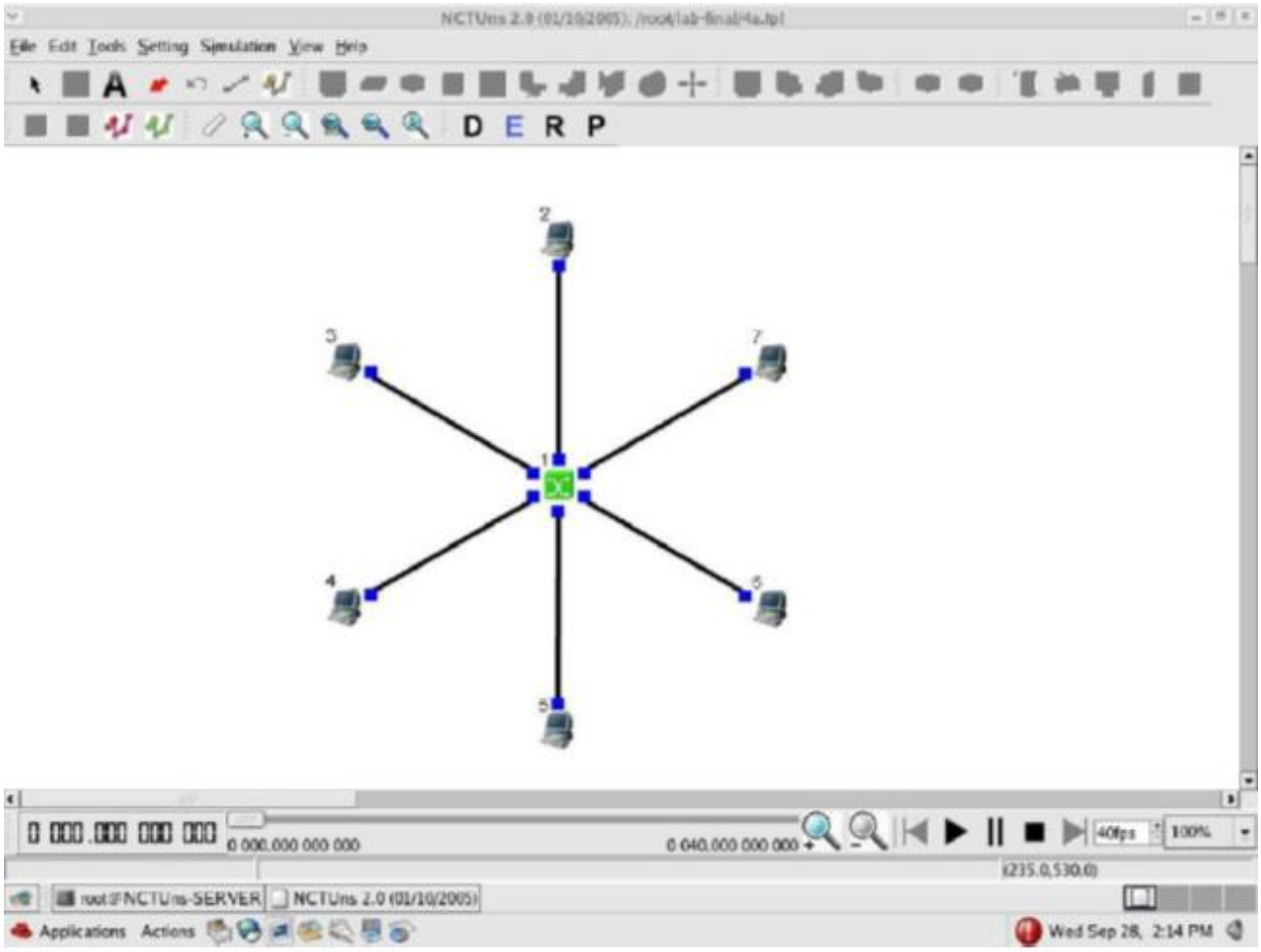
Step 4: Now run the simulation.

Step 5: Now click on any one of the hosts and click on command console and ping the destination node.

ping IP Address of the host

Note: The no: of drop packets are obtained only when the traffic is more in the network. For checking the no of packets dropped press ctrl+C

The screenshot of the topology is shown below:



vi) Simulate an Ethernet LAN using N nodes and set multiple traffic nodes and plot congestion window for different source/destination.

STEPS:

Step 1: Connect one set of hosts with a hub and another set of hosts also through a hub and connect these two hubs through a switch. This forms an Ethernet LAN.

Step 2: Setup multiple traffic connections between the hosts on one hub and hosts on another hub using the following command:

stcp [-p port] [-l writesize] hostIPaddr rtcp [-p port] [-l readsize]

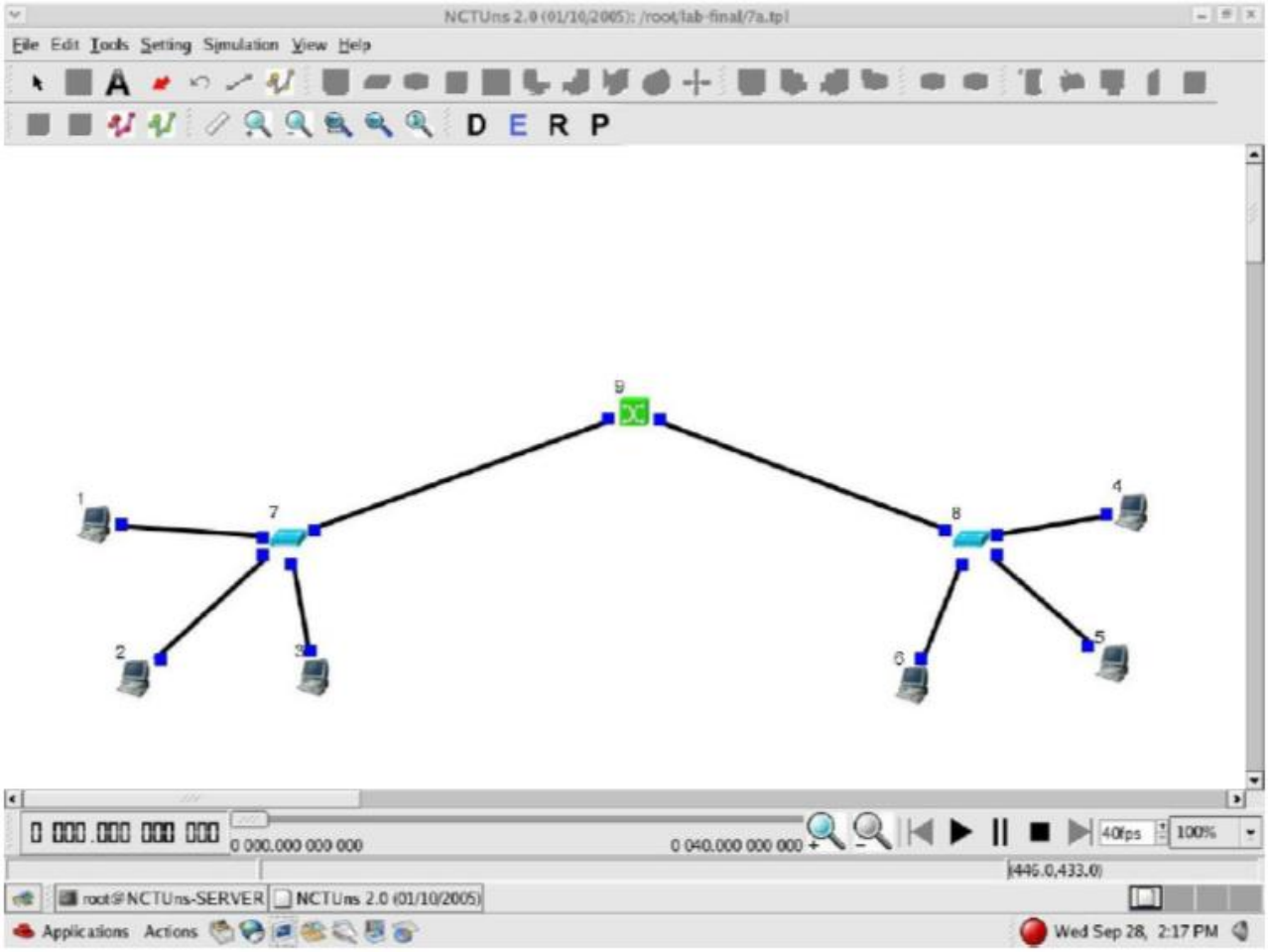
Step 3: Setup the collision log at the destination hosts in the MAC layer as described in the earlier

experiments.

Step 4: To plot the congestion window go to Menu->Tools->Plot Graph->File->open->filename.results->filename.coll.log

Step 5: View the results in the filename.results.

The screenshot of the topology is shown below:



vii) Simulate simple ESS and with transmitting nodes in wireless LAN by simulation and determine the performance with respect to transmission of packets.

STEPS:

Step 1: Connect a host and two WLAN access points to a router.

Step 2: Setup multiple mobile nodes around the two WLAN access points and set the path for each mobile node.

Step 3: Setup a ttcp connection between the mobile nodes and host using the following command:

Mobile Host 1

ttcp –t –u –s –p 3000 IPAddrOf Receiver

Mobile Host 1

ttcp –t –u –s –p 4000 IPAddrOf Receiver

Host(Receiver)

ttcp –r –u –s –p 3000 ttcp –r –u –s –p 4000

Step 4: Setup the input throughput log at the destination host.

Step 5: To set the transmission range go to Menu->Settings->WLAN mobile node- >Show transmission range.

Step 5: View the results in the filename. results.

Screenshot:

